# Spring 面试题

---

大家好，我是小林。

Spring 这块在 Java 后端面试里的地位毋庸置疑，几乎是必考内容。不管是校招还是社招，IOC、AOP、事务、Bean 生命周期这些主题，基本每次都逃不掉。很多人平时天天用 Spring 写代码，@Autowired 一打、@Transactional 一加就完事了，但真被问起"IOC 底层怎么实现的""循环依赖是怎么解决的""事务为什么会失效"，就开始语焉不详了。

这篇文章整理了 Spring 全家桶面试中最常被问到的知识点，涵盖 Spring 核心（IOC/AOP/事务/Bean）、SpringMVC、SpringBoot 自动装配、MyBatis，以及 SpringCloud 微服务组件，内容覆盖比较全面。

几个在面试里出现频率特别高的方向，建议重点花时间：

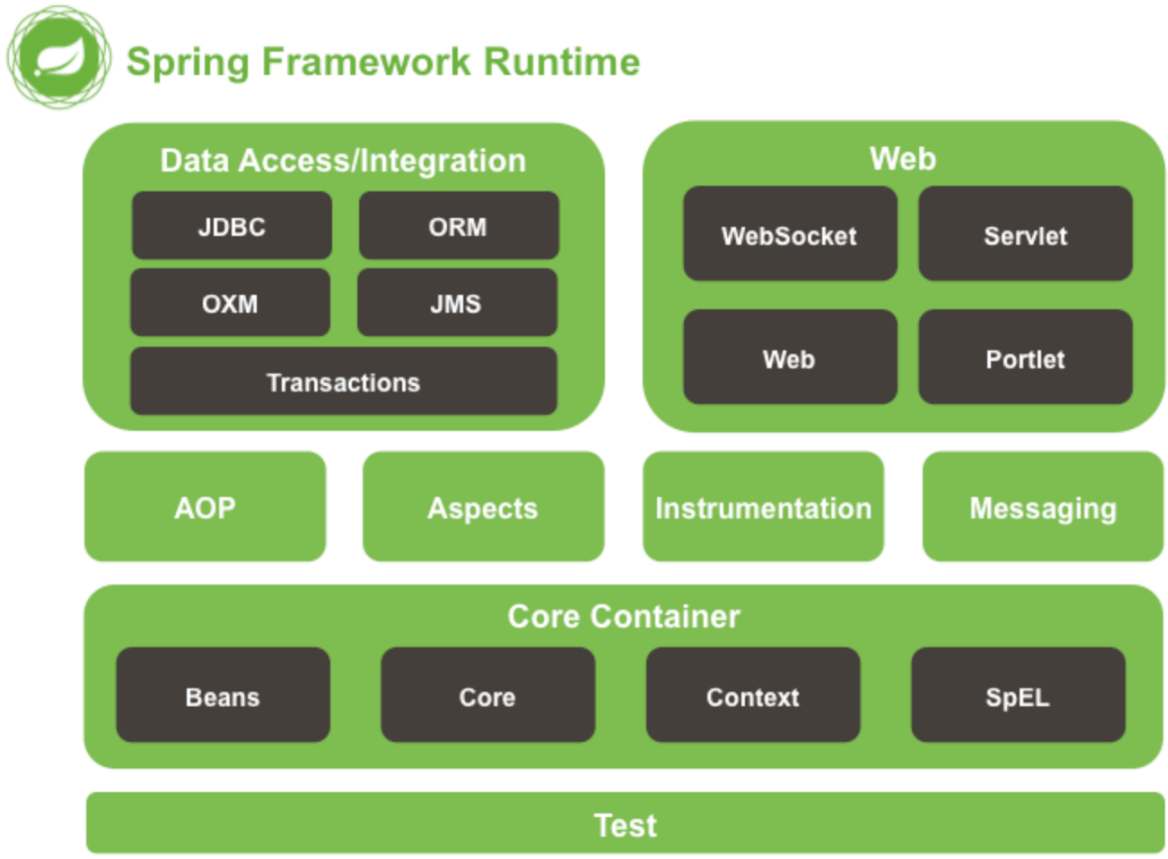
- **IOC 和 AOP**：IOC 的底层实现原理、三种依赖注入方式的区别，AOP 的动态代理机制（JDK 动态代理 vs CGLIB），以及 AOP 的实际应用场景，这两块是 Spring 面试的绝对核心。
- **循环依赖和三级缓存**：为什么需要三级缓存、二级缓存为什么不够，这个问题在面试里被问得越来越频繁，而且很容易被追问到细节。
- **Spring 事务**：事务什么时候会失效（用 this 调用、非 public 方法、异常类型不对），这是一道经典坑题，答起来很容易只说表面。
- **SpringBoot 自动装配原理**：`@SpringBootApplication` 背后的逻辑、`spring.factories` 是怎么工作的，这块是 SpringBoot 的核心，面试里问的很多。
- **Bean 的生命周期**：从实例化到销毁经历了哪些步骤、各个扩展点在哪里介入，这块答好了能体现对 Spring 底层的理解深度。

如果你是第一次系统准备 Spring，建议先搞清楚 IOC 和 AOP 的原理，再去看 Bean 生命周期和循环依赖，顺序理清楚之后，后面的 SpringBoot 自动装配理解起来会顺很多。

---

## Spring

### 说一下你对 Spring 的理解




Spring 框架核心特性包括：

- **IoC 容器**：Spring 通过控制反转实现了对象的创建和对象间的依赖关系管理。开发者只需要定义好 Bean 及其依赖关系，Spring 容器负责创建和组装这些对象。
- **AOP**：面向切面编程，允许开发者定义横切关注点，例如事务管理、安全控制等，独立于业务逻辑的代码。通过 AOP，可以将这些关注点模块化，提高代码的可维护性和可重用性。
- **事务管理**：Spring 提供了一致的事务管理接口，支持声明式和编程式事务。开发者可以轻松地进行事务管理，而无需关心具体的事务 API。
- **MVC 框架**：Spring MVC 是一个基于 Servlet API 构建的 Web 框架，采用了模型-视图-控制器（MVC）架构。它支持灵活的 URL 到页面控制器的映射，以及多种视图技术。

## spring 的核心思想说说你的理解?

| 核心思想 | 解决的问题 | 实现手段 | 典型应用场景 |
| --- | --- | --- | --- |
| IOC | 对象创建与依赖管理的高耦合 | 容器管理 Bean 生命周期 | 动态替换数据库实现、服务组装 |
| DI | 依赖关系的硬编码问题 | Setter/构造器/注解注入 | 注入数据源、服务层依赖 DAO 层 |
| AOP | 横切逻辑分散在业务代码中 | 动态代理与切面配置 | 日志、事务、权限校验统一处理 |

Spring 通过这 IOC、DI、AOP 三大核心思想，实现了轻量级、高内聚低耦合的企业级应用开发框架，成为 Java 生态中不可或缺的基石。

## Spring IoC 和 AOP 介绍一下

Spring IoC 和 AOP 区别：

- **IoC**：即控制反转的意思，它是一种创建和获取对象的技术思想，依赖注入(DI)是实现这种技术的一种方式。传统开发过程中，我们需要通过 new 关键字来创建对象。使用 IoC 思想开发方式的话，我们不通过 new 关键字创建对象，而是通过 IoC 容器来帮我们实例化对象。 通过 IoC 的方式，可以大大降低对象之间的耦合度。
- **AOP**：是面向切面编程，能够将那些与业务无关，却为业务模块所共同调用的逻辑封装起来，以减少系统的重复代码，降低模块间的耦合度。Spring AOP 就是基于动态代理的，如果要代理的对象，实现了某个接口，那么 Spring AOP 会使用 JDK Proxy，去创建代理对象，而对于没有实现接口的对象，就无法使用 JDK Proxy 去进行代理了，这时候 Spring AOP 会使用 Cglib 生成一个被代理对象的子类来作为代理。

在 Spring 框架中，IOC 和 AOP 结合使用，可以更好地实现代码的模块化和分层管理。例如：

- 通过 IOC 容器管理对象的依赖关系，然后通过 AOP 将横切关注点统一切入到需要的业务逻辑中。

- 使用 IOC 容器管理 Service 层和 DAO 层的依赖关系，然后通过 AOP 在 Service 层实现事务管理、日志记录等横切功能，使得业务逻辑更加清晰和可维护。

## Spring 的 aop 介绍一下

Spring AOP 是 Spring 框架中的一个重要模块，用于实现面向切面编程。

我们知道，Java 就是一门面向对象编程的语言，在 OOP 中最小的单元就是“Class 对象”，但是在 AOP 中最小的单元是“切面”。一个“切面”可以包含很多种类型和对象，对它们进行模块化管理，例如事务管理。

在面向切面编程的思想里面，把功能分为两种

- **核心业务**：登陆、注册、增、删、改、查、都叫核心业务
- **周边功能**：日志、事务管理这些次要的为周边业务

在面向切面编程中，核心业务功能和周边功能是分别独立进行开发，两者不是耦合的，然后把切面功能和核心业务功能 "编织" 在一起，这就叫 AOP。

AOP 能够将那些与业务无关，**却为业务模块所共同调用的逻辑或责任（例如事务处理、日志管理、权限控制等）封装起来**，便于**减少系统的重复代码，降低模块间的耦合度**，并**有利于未来的可拓展性和可维护性**。

在 AOP 中有以下几个概念：

- **AspectJ**：切面，只是一个概念，没有具体的接口或类与之对应，是 Join point，Advice 和 Pointcut 的一个统称。
- **Join point**：连接点，指程序执行过程中的一个点，例如方法调用、异常处理等。在 Spring AOP 中，仅支持方法级别的连接点。
- **Advice**：通知，即我们定义的一个切面中的横切逻辑，有“around”，“before”和“after”三种类型。在很多的 AOP 实现框架中，Advice 通常作为一个拦截器，也可以包含许多个拦截器作为一条链路围绕着 Join point 进行处理。
- **Pointcut**：切点，用于匹配连接点，一个 AspectJ 中包含哪些 Join point 需要由 Pointcut 进行筛选。
- **Introduction**：引介，让一个切面可以声明被通知的对象实现任何他们没有真正实现的额外的接口。例如可以让一个代理对象代理两个目标类。

- **Weaving**：织入，在有了连接点、切点、通知以及切面，如何将它们应用到程序中呢？没错，就是织入，在切点的引导下，将通知逻辑插入到目标方法上，使得我们的通知逻辑在方法调用时得以执行。
- **AOP proxy**：AOP 代理，指在 AOP 实现框架中实现切面协议的对象。在 Spring AOP 中有两种代理，分别是 JDK 动态代理和 CGLIB 动态代理。
- **Target object**：目标对象，就是被代理的对象。

Spring AOP 是基于 JDK 动态代理和 Cglib 提升实现的，两种代理方式都属于运行时的一个方式，所以它没有编译时的一个处理，那么因此 Spring 是通过 Java 代码实现的。

## IOC 和 AOP 是通过什么机制来实现的?

> Spring IOC 实现机制

- **反射**：Spring IOC 容器利用 Java 的反射机制动态地加载类、创建对象实例及调用对象方法，反射允许在运行时检查类、方法、属性等信息，从而实现灵活的对象实例化和管理。
- **依赖注入**：IOC 的核心概念是依赖注入，即容器负责管理应用程序组件之间的依赖关系。Spring 通过构造函数注入、属性注入或方法注入，将组件之间的依赖关系描述在配置文件中或使用注解。
- **设计模式 - 工厂模式**：Spring IOC 容器通常采用工厂模式来管理对象的创建和生命周期。容器作为工厂负责实例化 Bean 并管理它们的生命周期，将 Bean 的实例化过程交给容器来管理。
- **容器实现**：Spring IOC 容器是实现 IOC 的核心，通常使用 BeanFactory 或 ApplicationContext 来管理 Bean。BeanFactory 是 IOC 容器的基本形式，提供基本的 IOC 功能；ApplicationContext 是 BeanFactory 的扩展，并提供更多企业级功能。

> Spring AOP 实现机制

Spring AOP 的实现依赖于**动态代理技术**。动态代理是在运行时动态生成代理对象，而不是在编译时。它允许开发者在运行时指定要代理的接口和行为，从而实现在不修改源码的情况下增强方法的功能。

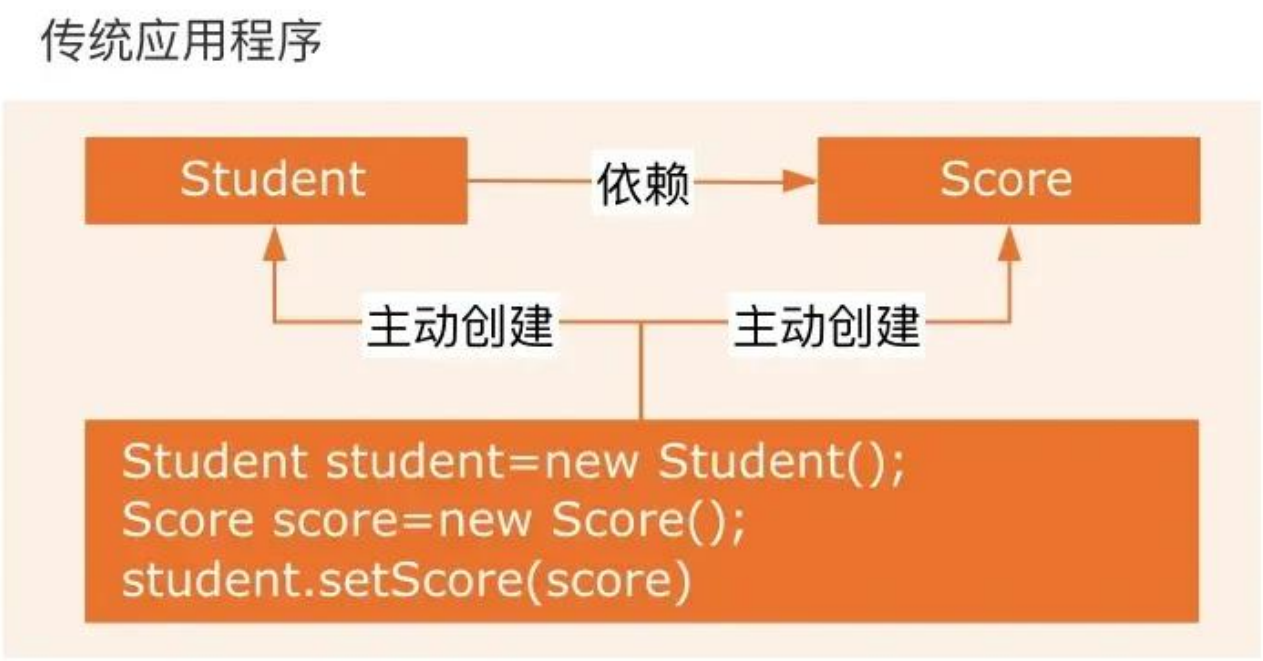
Spring AOP 支持两种动态代理：

- **基于 JDK 的动态代理**：使用 java.lang.reflect.Proxy 类和 java.lang.reflect.InvocationHandler 接口实现。这种方式需要代理的类实现一个或多个接口。

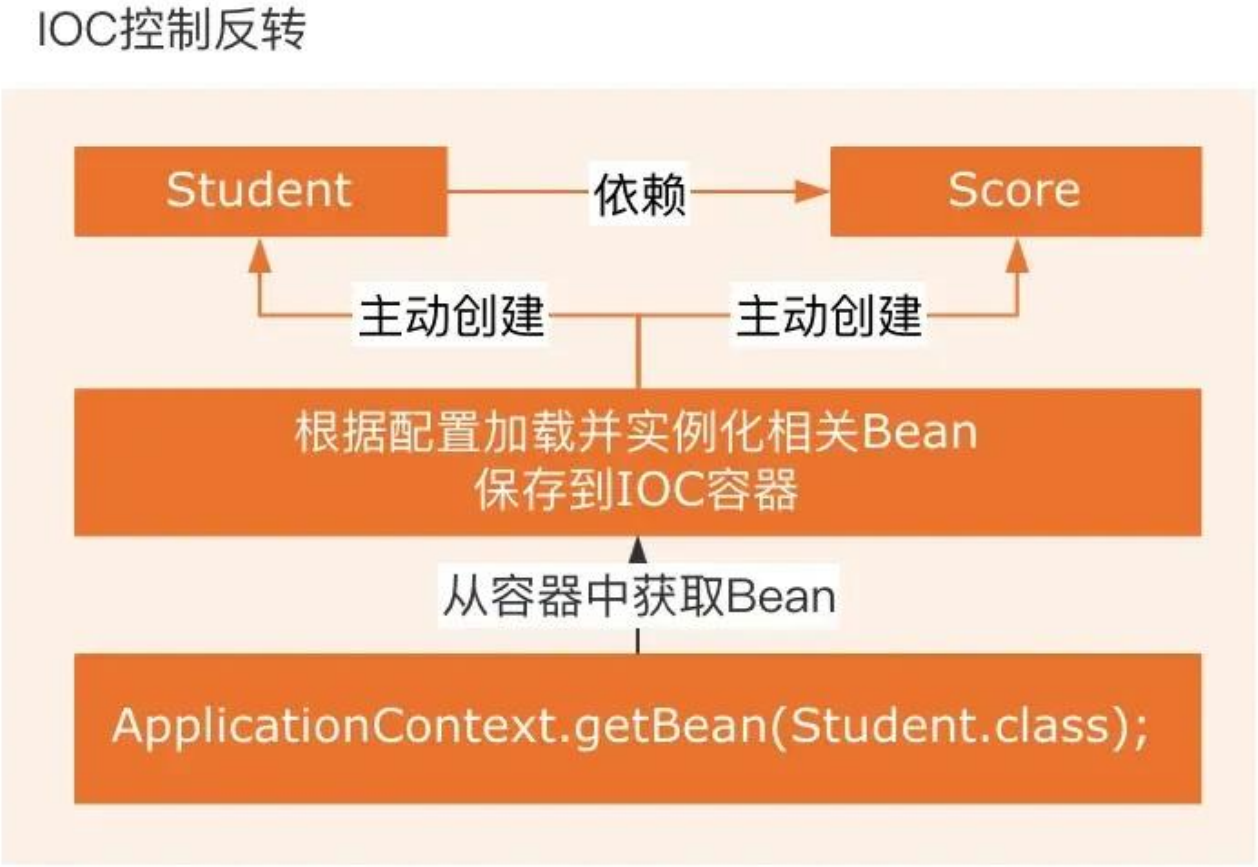
- **基于 CGLIB 的动态代理：**当被代理的类没有实现接口时，Spring 会使用 CGLIB 库生成一个被代理类的子类作为代理。CGLIB（Code Generation Library）是一个第三方代码生成库，通过继承方式实现代理。

## 怎么理解 SpringIoc?

IOC：Inversion Of Control，即控制反转，是一种设计思想。在传统的 Java SE 程序设计中，我们直接在对象内部通过 new 的方式来创建对象，是程序主动创建依赖对象；




而在 Spring 程序设计中，IOC 是有专门的容器去控制对象。




**所谓控制**就是对象的创建、初始化、销毁。

- 创建对象：原来是 new 一个，现在是由 Spring 容器创建。
- 初始化对象：原来是对象自己通过构造器或者 setter 方法给依赖的对象赋值，现在是由 Spring 容器自动注入。
- 销毁对象：原来是直接给对象赋值 null 或做一些销毁操作，现在是 Spring 容器管理生命周期负责销毁对象。

总结：IOC 解决了繁琐的对象生命周期的操作，解耦了我们的代码。**所谓反转**：其实是反转的控制权，前面提到是由 Spring 来控制对象的生命周期，那么对象的控制就完全脱离了我们的控制，控制权交给了 Spring 。这个反转是指：我们由对象的控制者变成了 IOC 的被动控制者。

## 依赖倒置，依赖注入，控制反转分别是什么?

- 控制反转：“控制”指的是对程序执行流程的控制，而“反转”指的是在没有使用框架之前，程序员自己控制整个程序的执行。在使用框架之后，整个程序的执行流程通过框架来控制。流程的控制权从程序员“反转”给了框架。
- 依赖注入：依赖注入和控制反转恰恰相反，它是一种具体的编码技巧。我们不通过 new 的方式在类内部创建依赖类的对象，而是将依赖的类对象在外部创建好之后，通过构造函数、函数参数等方式传递（或注入）给类来使用。

- 依赖倒置：这条原则跟控制反转有点类似，主要用来指导框架层面的设计。高层模块不依赖低层模块，它们共同依赖同一个抽象。抽象不要依赖具体实现细节，具体实现细节依赖抽象。

## 依赖注入了解吗？怎么实现依赖注入的？

在传统编程中，当一个类需要使用另一个类的对象时，通常会在该类内部通过`new`关键字来创建依赖对象，这使得类与类之间的耦合度较高。

而依赖注入则是将对象的创建和依赖关系的管理交给 Spring 容器来完成，类只需要声明自己所依赖的对象，容器会在运行时将这些依赖对象注入到类中，从而降低了类与类之间的耦合度，提高了代码的可维护性和可测试性。

具体到 Spring 中，常见的依赖注入的实现方式，比如构造器注入、Setter 方法注入，还有字段注入。

- **构造器注入：**通过构造函数传递依赖对象，保证对象初始化时依赖已就绪。

```
@Service
public class UserService {
    private final UserRepository userRepository;

    // 构造器注入（Spring 4.3+ 自动识别单构造器，无需显式@Autowired）
    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }
}
```

- **Setter 方法注入：**通过 Setter 方法设置依赖，灵活性高，但依赖可能未完全初始化。

```
public class PaymentService {
    private PaymentGateway gateway;

    @Autowired
    public void setGateway(PaymentGateway gateway) {
        this.gateway = gateway;
    }
}
```

- **字段注入：**直接通过 `@Autowired` 注解字段，代码简洁但隐藏依赖关系，不推荐生产代码。

```
@Service
public class OrderService {
    @Autowired
    private OrderRepository orderRepository;
}
```

## 如果让你设计一个 SpringIoc，你觉得会从哪些方面考虑这个设计?

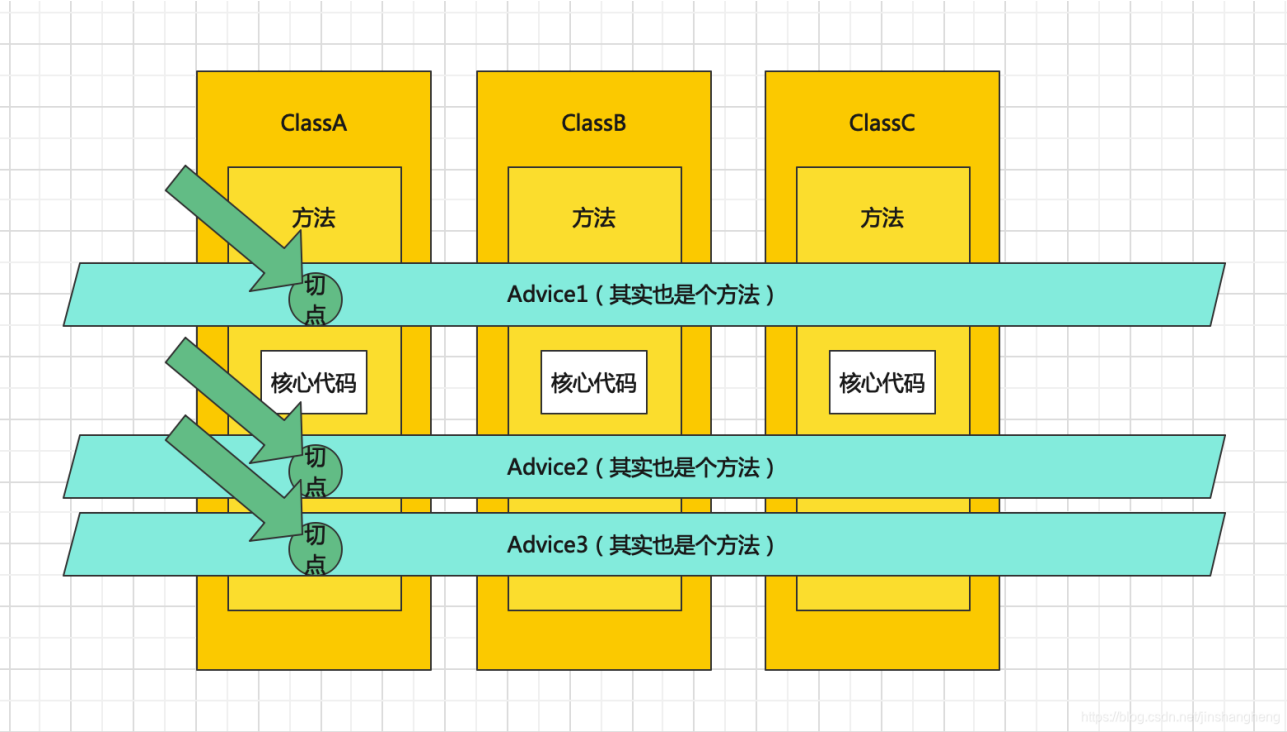
- Bean 的生命周期管理：需要设计 Bean 的创建、初始化、销毁等生命周期管理机制，可以考虑使用工厂模式和单例模式来实现。
- 依赖注入：需要实现依赖注入的功能，包括属性注入、构造函数注入、方法注入等，可以考虑使用反射机制和 XML 配置文件来实现。
- Bean 的作用域：需要支持多种 Bean 作用域，比如单例、原型、会话、请求等，可以考虑使用 Map 来存储不同作用域的 Bean 实例。
- AOP 功能的支持：需要支持 AOP 功能，可以考虑使用动态代理机制和切面编程来实现。
- 异常处理：需要考虑异常处理机制，包括 Bean 创建异常、依赖注入异常等，可以考虑使用 try-catch 机制来处理异常。
- 配置文件加载：需要支持从不同的配置文件中加载 Bean 的相关信息，可以考虑使用 XML、注解或者 Java 配置类来实现。

## SpringAOP 主要想解决什么问题

它的目的是对于面向对象思维的一种补充，而不是像引入命令式、函数式编程思维让他顺应另一种开发场景。在我个人的理解下 AOP 更像是一种对于不支持多继承的弥补，除开对象的主要特征（我更喜欢叫“强共性”）被抽象为了一条继承链路，对于一些“弱共性”，AOP 可以统一对他们进行抽象和集中处理。

举一个简单的例子，打印日志。需要打印日志可能是许多对象的一个共性，这在企业级开发中十分常见，但是日志的打印并不反应这个对象的主要共性。而日志的打印又是一个具体的内容，它并不抽象，所以它的工作也不可以用接口来完成。而如果利用继承，打印日志的工作又横跨继承树下面的多个同级子节点，强行侵入到继承树内进行归纳会干扰这些强共性的区分。

这时候，我们就需要 AOP 了。AOP 首先在一个 Aspect（切面）里定义了一些 Advice（增强），其中包含具体实现的代码，同时整理了切入点，切入点的粒度是方法。最后，我们将这些 Advice 织入到对象的方法上，形成了最后执行方法时面对的完整方法。




## AOP 在 spring 中的应用，你知道哪些?

在 Spring 中，AOP（面向切面编程）是核心特性之一，它通过分离横切关注点（如日志、事务、安全等）与业务逻辑，让代码更清晰、可维护。实际应用中，Spring AOP 的典型场景有这么几个：

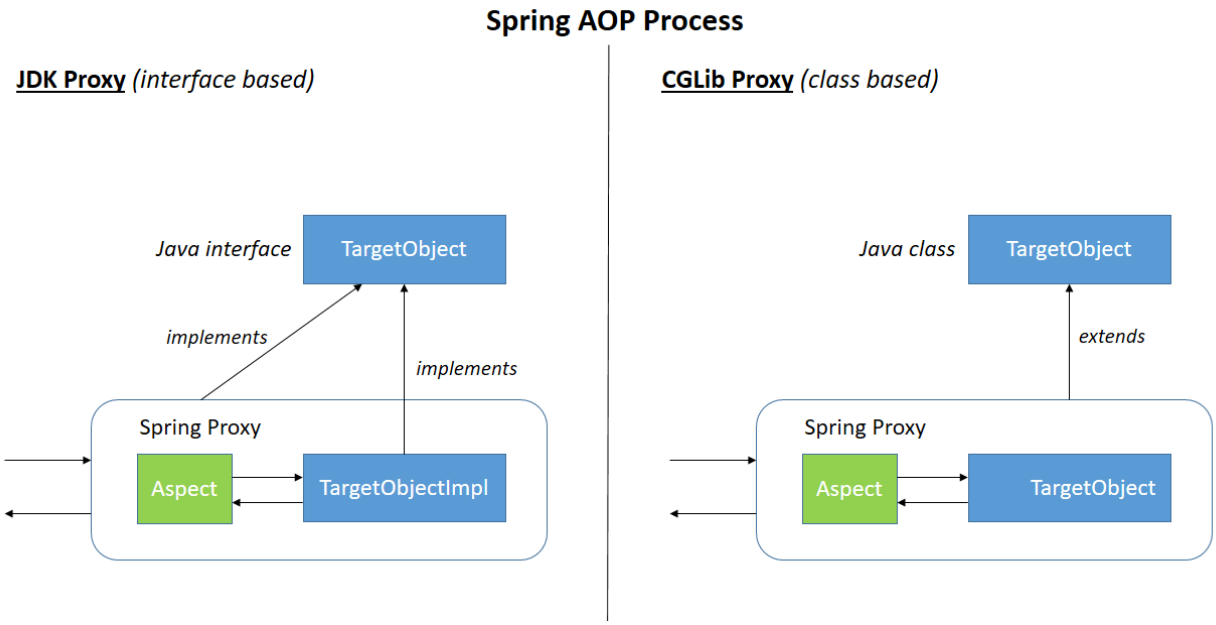
- 最常见的是**事务管理**，Spring 的声明式事务就是基于 AOP 实现的。我们只需要在方法上标注`@Transactional`，Spring 就会通过 AOP 在方法执行前开启事务，执行后根据是否有异常决定提交或回滚，不用手动写事务控制代码，大大简化了事务管理。
- 然后是**日志记录**，比如对接口的调用参数、返回结果、执行时间做日志，用 AOP 可以把这些逻辑抽离出来，定义一个切面，通过`@Before`获取入参，`@AfterReturning`获取返回值，`@Around`统计执行时间，这样业务方法里就不用掺杂日志代码，既干净又方便统一修改。
- 还有**权限校验**，比如某些接口需要登录后才能访问，或者需要特定角色权限。可以定义一个切面，在方法执行前（`@Before`）检查用户的登录状态或权限，如果不满足就直接抛出异常阻止方法执行，避免在每个接口方法里重复写权限判断。

Spring AOP 的这些应用，本质上都是通过 “切面” 封装横切逻辑，通过 “切入点” 指定要作用的方法，然后在方法执行的不同阶段（前、后、异常时等）插入这些逻辑，实现了 “无侵入” 地增强业务功能，这也是 AOP 在 Spring 中如此重要的原因。

## 谈谈你对 Spring AOP 的了解程度?

我对 Spring AOP 的理解，核心就是它是一种**不侵入业务代码的横切逻辑解决方案**。比如日志、事务、权限校验这些，如果全写在业务方法里会非常乱，AOP 的作用就是把这些和业务无关、但又必须要做的事情抽出来，统一处理。

在 Spring 里，AOP 主要是通过**代理机制**来实现的。默认情况下，如果目标类实现了接口，Spring 用的是 JDK 动态代理；如果没有接口，就用 CGLIB 生成子类代理。所以本质上，调用的并不是原始对象，而是代理对象，方法执行前后会被织入额外逻辑。




从使用角度来看，Spring AOP 是**基于运行时代理的 AOP**，不是像 AspectJ 那种编译期或类加载期织入。我们平时写 `@Aspect`、`@Before`、`@After`、`@Around` 这些注解，其实就是在定义切面、切点和通知，Spring 在容器启动的时候会解析这些配置，最后帮我们把代理对象准备好。

真正执行的时候，方法调用会走一条拦截器链。以 `@Around` 为例，本质上就是在方法执行前后包一层逻辑，是否放行、是否执行原方法，都是通过 `proceed()` 来控制的。事务、日志这些底层实现，其实就是一组 MethodInterceptor 在按顺序执行。

在实际项目里，我主要用 AOP 来做几类事情，比如声明式事务、接口日志、统一异常处理、参数校验或者一些轻量级的权限控制。这些场景的共同点是，它们都不属于核心业务逻辑，但又需要在很多地方重复出现，用 AOP 抽出来会非常干净。

当然 Spring AOP 也有它的边界。第一是它**只能作用在 Spring 容器管理的 Bean 上**，自己 new 出来的对象是不会生效的。第二是**同类内部方法调用不会触发 AOP**，因为走的是 `this`，绕过了代理。第三是它只能拦截方法级别的调用，做不到字段级别或者构造方法级别的增强。

整体来说，我会把 Spring AOP 当成一种**工程层面的工具**，用来解决通用、可复用的横切问题，而不是拿它去写复杂业务逻辑，超过这个边界我一般会考虑别的设计方式。

## SpringAOP 的原理了解吗

Spring AOP 的实现依赖于**动态代理技术**。动态代理是在运行时动态生成代理对象，而不是在编译时。它允许开发者在运行时指定要代理的接口和行为，从而实现在不修改源码的情况下增强方法的功能。

Spring AOP 支持两种动态代理：

- **基于 JDK 的动态代理**：使用 java.lang.reflect.Proxy 类和 java.lang.reflect.InvocationHandler 接口实现。这种方式需要代理的类实现一个或多个接口。
- **基于 CGLIB 的动态代理**：当被代理的类没有实现接口时，Spring 会使用 CGLIB 库生成一个被代理类的子类作为代理。CGLIB（Code Generation Library）是一个第三方代码生成库，通过继承方式实现代理。

## 动态代理是什么?

Java 的动态代理是一种在运行时动态创建代理对象的机制，主要用于在不修改原始类的情况下对方法调用进行拦截和增强。

Java 动态代理主要分为两种类型：

- **基于接口的代理**（JDK 动态代理）： 这种类型的代理要求目标对象必须实现至少一个接口。Java 动态代理会创建一个实现了相同接口的代理类，然后在运行时动态生成该类的实例。这种代理的实现核心是 `java.lang.reflect.Proxy` 类和 `java.lang.reflect.InvocationHandler` 接口。每一个动态代理类都必须实现 `InvocationHandler` 接口，并且每个代理类的实例都关联到一个 `handler`。当通过代理对象调用一个方法时，这个方法的调用会被转发为由 `InvocationHandler` 接口的 `invoke()`方法来进行调用。
- **基于类的代理**（CGLIB 动态代理）： CGLIB（Code Generation Library）是一个强大的高性能的代码生成库，它可以在运行时动态生成一个目标类的子类。CGLIB 代理不需要目标类实现接口，而是通过继承的方式创建代理类。因此，如果目标对象没有实现任何接口，可以使用 CGLIB 来创建动态代理。

## 动态代理和静态代理的区别

代理是一种常用的设计模式，目的是：为其他对象提供一个代理以控制对某个对象的访问，将两个类的关系解耦。代理类和委托类都要实现相同的接口，因为代理真正调用的是委托类的方法。

区别：

- 静态代理：由程序员创建或者是由特定工具创建，在代码编译时就确定了被代理的类是一个静态代理。静态代理通常只代理一个类；
- 动态代理：在代码运行期间，运用反射机制动态创建生成。动态代理代理的是一个接口下的多个实现类。

## 能使用静态代理的方式实现 AOP 吗?

当然**可以**用静态代理实现 AOP 的基本功能，比如你在代码里手动写个代理类，在目标方法前后加日志或者事务控制，技术上完全可行。但问题是，静态代理在实际生产中基本没人用，因为它有三大硬伤：

- **第一是代码爆炸**。比如你有 100 个 Service 类需要加事务，就得写 100 个对应的静态代理类，里面全是重复的 `try-catch` 提交回滚代码，维护起来简直是灾难。
- **第二是僵化**。一旦业务接口改了个方法名，所有相关的代理类都得跟着改，而动态代理通过反射调用目标方法，根本不怕这种变动。
- **第三是无法动态筛选**。比如你想只给带 `@Transactional` 注解的方法加事务，静态代理只能写死逻辑，而 Spring AOP 可以在运行时通过切点表达式精准匹配需要增强的方法。

所以 Spring 才选了 JDK 动态代理和 CGLIB：它们能在**运行时动态生成代理类**，一个切面配置就能覆盖成百上千个方法。像事务管理这种全局需求，用静态代理手动绑定根本不可行。

## AOP 实现有哪些注解?

常用的注解包括：

- @Aspect：用于定义切面，标注在切面类上。
- @Pointcut：定义切点，标注在方法上，用于指定连接点。
- @Before：在方法执行之前执行通知。
- @After：在方法执行之后执行通知。
- @Around：在方法执行前后都执行通知。
- @AfterReturning：在方法执行后返回结果后执行通知。
- @AfterThrowing：在方法抛出异常后执行通知。

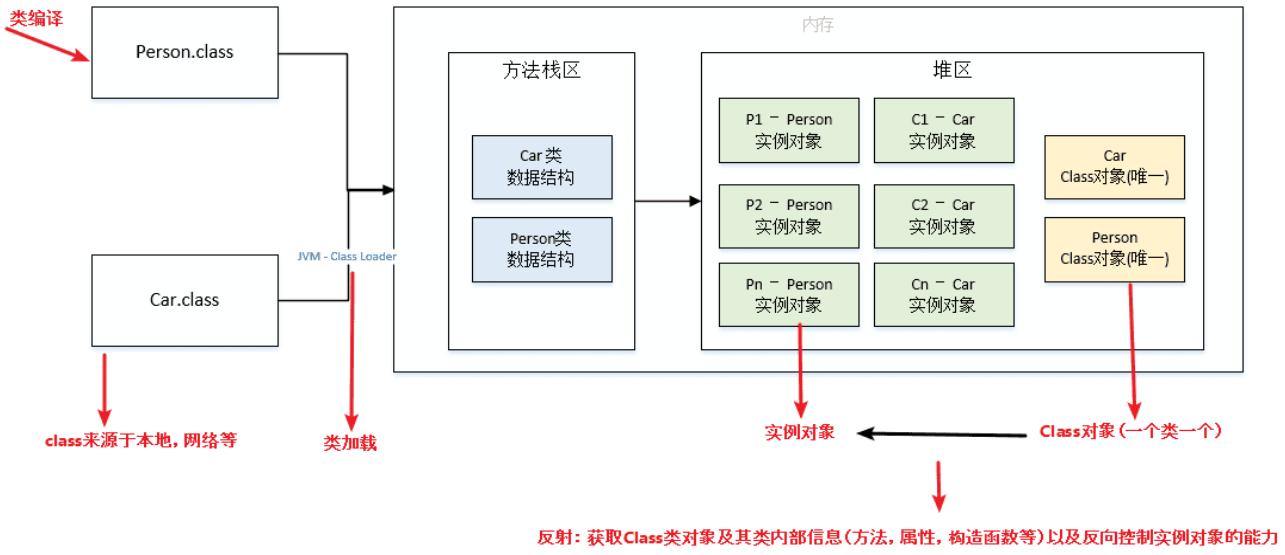
- @Advice：通用的通知类型，可以替代@Before、@After 等。

## 什么是反射？有哪些使用场景？

反射机制是指程序在运行状态下，对于任意一个类，都能够获取这个类的所有属性和方法；对于任意一个对象，都能够调用它的任意属性和方法。也就是说，Java 反射允许在运行时获取类的信息并动态操作对象，即使在编译时不知道具体的类也能实现。

反射具有以下特性：

1. **运行时类信息访问**：反射机制允许程序在运行时获取类的完整结构信息，包括类名、包名、父类、实现的接口、构造函数、方法和字段等。
2. **动态对象创建**：可以使用反射 API 动态地创建对象实例，即使在编译时不知道具体的类名。这是通过 Class 类的 newInstance()方法或 Constructor 对象的 newInstance()方法实现的。
3. **动态方法调用**：可以在运行时动态地调用对象的方法，包括私有方法。这通过 Method 类的 invoke()方法实现，允许你传入对象实例和参数值来执行方法。
4. **访问和修改字段值**：反射还允许程序在运行时访问和修改对象的字段值，即使是私有的。这是通过 Field 类的 get()和 set()方法完成的。




Java 反射机制在 spring 框架中，很多地方都用到了反射，让我们来看看 Spring 的 IoC 和 AOP 是如何使用反射技术的。

> 1、Spring 框架的依赖注入（DI）和控制反转（IoC）

Spring 使用反射来实现其核心特性：依赖注入。

在 Spring 中，开发者可以通过 XML 配置文件或者基于注解的方式声明组件之间的依赖关系。当应用程序启动时，Spring 容器会扫描这些配置或注解，然后利用反射来实例化 Bean（即 Java 对象），并根据配置自动装配它们的依赖。

例如，当一个 Service 类需要依赖另一个 DAO 类时，开发者可以在 Service 类中使用@Autowired 注解，而无需自己编写创建 DAO 实例的代码。Spring 容器会在运行时解析这个注解，通过反射找到对应的 DAO 类，实例化它，并将其注入到 Service 类中。这样不仅降低了组件之间的耦合度，也极大地增强了代码的可维护性和可测试性。

> 2、动态代理的实现

在需要对现有类的方法调用进行拦截、记录日志、权限控制或是事务管理等场景中，反射结合动态代理技术被广泛应用。

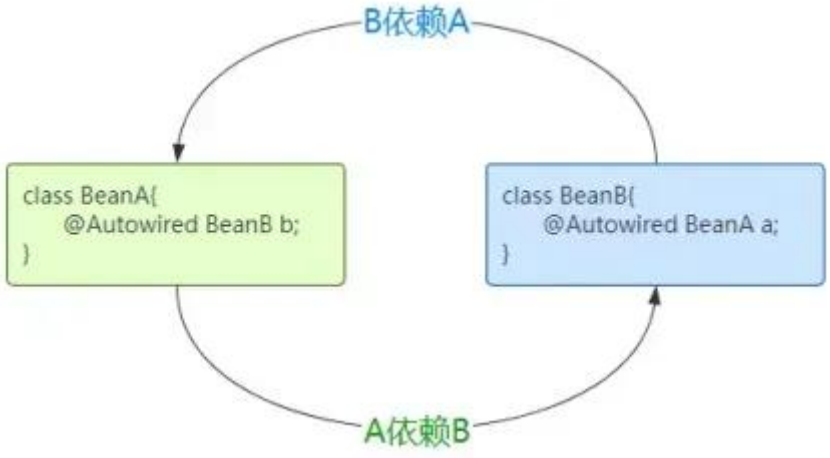
一个典型的例子是 Spring AOP（面向切面编程）的实现。Spring AOP 允许开发者定义切面（Aspect），这些切面可以横切关注点（如日志记录、事务管理），并将其插入到业务逻辑中，而不需要修改业务逻辑代码。

例如，为了给所有的服务层方法添加日志记录功能，可以定义一个切面，在这个切面中，Spring 会使用 JDK 动态代理或 CGLIB（如果目标类没有实现接口）来创建目标类的代理对象。这个代理对象在调用任何方法前或后，都会执行切面中定义的代码逻辑（如记录日志），而这一切都是在运行时通过反射来动态构建和执行的，无需硬编码到每个方法调用中。

这两个例子展示了反射机制如何在实际工程中促进**松耦合、高内聚**的设计，以及如何提供动态、灵活的编程能力，特别是在框架层面和解决跨切面问题时。

## spring 是如何解决循环依赖的?

循环依赖指的是两个类中的属性相互依赖对方：例如 A 类中有 B 属性，B 类中有 A 属性，从而形成了一个依赖闭环，如下图。




循环依赖问题在 Spring 中主要有三种情况：

- 第一种：通过构造方法进行依赖注入时产生的循环依赖问题。
- 第二种：通过 setter 方法进行依赖注入且是在多例（原型）模式下产生的循环依赖问题。
- 第三种：通过 setter 方法进行依赖注入且是在单例模式下产生的循环依赖问题。

只有【第三种方式】的循环依赖问题被 Spring 解决了，其他两种方式在遇到循环依赖问题时，Spring 都会产生异常。

Spring 在 `DefaultSingletonBeanRegistry` 类中维护了三个重要的缓存 (Map)，称为“三级缓存”：

- `singletonObjects` (一级缓存)：存放的是完全初始化好的、可用的 Bean 实例，`getBean()` 方法最终返回的就是这里面的 Bean。此时 Bean 已实例化、属性已填充、初始化方法已执行、AOP 代理（如果需要）也已生成。
- `earlySingletonObjects` (二级缓存)：存放的是提前暴露的 Bean 的原始对象引用 或 早期代理对象引用，专门用来处理循环依赖。当一个 Bean 还在创建过程中（尚未完成属性填充和初始化），但它的引用需要被注入到另一个 Bean 时，就暂时放在这里。此时 Bean 已实例化（调用了构造函数），但属性尚未填充，初始化方法尚未执行，它可能是一个原始对象，也可能是一个为了解决 AOP 代理问题而提前生成的代理对象。
- `singletonFactories` (三级缓存)：存放的是 Bean 的 `ObjectFactory` 工厂对象。，这是解决循环依赖和 AOP 代理协同工作的关键。当 Bean 被实例化后（刚调完构造函数），Spring 会创建一个 `ObjectFactory` 并将其放入三级缓存。这个工厂的 `getObject()` 方法负责返回该 Bean 的早期引用（可能是原始对象，也可能是提前生成的代理对象），当检测到循环依赖需要注入一个尚未完全初始化的 Bean 时，就会调用这个工厂来获取早期引用。

Spring 通过 三级缓存 和 提前暴露未完全初始化的对象引用 的机制来解决单例作用域 Bean 的 sette 注入方式的循环依赖问题。

假设存在两个相互依赖的单例 Bean：`BeanA` 依赖 `BeanB`，同时 `BeanB` 也依赖 `BeanA`。当 Spring 容器启动时，它会按照以下流程处理：

- 第一步：创建 `BeanA` 的实例并提前暴露工厂。

Spring 首先调用 `BeanA` 的构造函数进行实例化，此时得到一个原始对象（尚未填充属性）。紧接着，Spring 会将一个特殊的 `ObjectFactory` 工厂对象存入第三级缓存（`singletonFactories`）。这个工厂的使命是：当其他 Bean 需要引用 `BeanA` 时，它能动态返回当前这个半成品的 `BeanA`（可能是原始对象，也可能是为应对 AOP 而提前生成的代理对象）。此时 `BeanA` 的状态是"已实例化但未初始化"，像一座刚搭好钢筋骨架的大楼。

- 第二步：填充 `BeanA` 的属性时触发 `BeanB` 的创建。

Spring 开始为 `BeanA` 注入属性，发现它依赖 `BeanB`。于是容器转向创建 `BeanB`，同样先调用其构造函数实例化，并将 `BeanB` 对应的 `ObjectFactory` 工厂存入三级缓存。至此，三级缓存中同时存在 `BeanA` 和 `BeanB` 的工厂，它们都代表未完成初始化的半成品。

- 第三步：`BeanB` 属性注入时发现循环依赖。

当 Spring 试图填充 `BeanB` 的属性时，检测到它需要注入 `BeanA`。此时容器启动依赖查找：

1. 在一级缓存（存放完整 Bean）中未找到 `BeanA`；
2. 在二级缓存（存放已暴露的早期引用）中同样未命中；
3. 最终在三级缓存中定位到 `BeanA` 的工厂。

Spring 立即调用该工厂的 `getObject()`方法。这个方法会执行关键决策：若 `BeanA` 需要 AOP 代理，则动态生成代理对象（即使 `BeanA` 还未初始化）；若无需代理，则直接返回原始对象。得到的这个早期引用（可能是代理）被放入二级缓存（`earlySingletonObjects`），同时从三级缓存清理工厂条目。最后，Spring 将这个早期引用注入到 `BeanB` 的属性中。至此，`BeanB` 成功持有 `BeanA` 的引用——尽管 `BeanA` 此时仍是个半成品。

- 第四步：完成 `BeanB` 的生命周期。

`BeanB` 获得所有依赖后，Spring 执行其初始化方法（如`@PostConstruct`），将其转化为完整可用的 Bean。随后，`BeanB` 被提升至一级缓存（`singletonObjects`），二级和三级缓存中关于 `BeanB` 的临时条目均被清除。此时 `BeanB` 已准备就绪，可被其他对象使用。

- 第五步：回溯完成 `BeanA` 的构建。

随着 `BeanB` 创建完毕，流程回溯到最初中断的 `BeanA` 属性注入环节。Spring 将已完备的 `BeanB` 实例注入 `BeanA`，接着执行 `BeanA` 的初始化方法。这里有个精妙细节：若之前为 `BeanA` 生成过早期代理，

Spring 会直接复用二级缓存中的代理对象作为最终 Bean，而非重复创建。最终，完全初始化的 `BeanA`（可能是原始对象或代理）入驻一级缓存，其早期引用从二级缓存移除。至此循环闭环完成，两个 Bean 皆可用。

三级缓存的设计的精髓：

- **三级缓存工厂**（`singletonFactories`）负责在实例化后立刻暴露对象生成能力，兼顾 AOP 代理的提前生成；
- **二级缓存**（`earlySingletonObjects`）临时存储已确定的早期引用，避免重复生成代理；
- **一级缓存**（`singletonObjects`）最终交付完整 Bean。

整个机制通过**中断初始化流程、逆向注入半成品、延迟代理生成**三大策略，将循环依赖的死结转化为有序的接力协作。

值得注意的是，此方案仅适用于 Setter/Field 注入的**单例 Bean**；构造器注入因必须在实例化前获得依赖，仍会导致无解的死锁。

## Spring 为什么用 3 级缓存解决循环依赖问题？用 2 级缓存不行吗？

Spring 必须用三级缓存解决循环依赖，核心是为了**正确处理需要 AOP 代理的 Bean**。如果只用二级缓存，会导致注入的对象形态错误，甚至破坏单例原则。

举个例子：假设 Bean A 依赖 B，B 又依赖 A，且 A 需要被动态代理（比如加了 `@Transactional`）。如果只有二级缓存，当 B 创建时去注入 A，拿到的是 A 的原始对象。但 A 在后续初始化完成后才会生成代理对象，结果就是：B 拿着原始对象 A，而 Spring 容器里存的是代理对象 A —— 同一个 Bean 出现了两个不同实例，这直接违反了单例的核心约束。

三级缓存中的 `ObjectFactory` 就是解决这个问题的关键。它不是直接缓存对象，而是存了一个能生产对象的工厂。当发生循环依赖时，调用这个工厂的 `getObject()` 方法，这时 Spring 会智能判断：如果这个 Bean 最终需要代理，就提前生成代理对象并放入二级缓存；如果不需要代理，就返回原始对象。这样一来，B 注入的 A 就是最终形态（可能是代理对象），后续 A 初始化完成后也不会再创建新代理，保证了对象全局唯一。

简单说，三级缓存的本质是 **“按需延迟生成正确引用”** 。它既维持了 Bean 生命周期的完整性（正常流程在初始化后生成代理），又在循环依赖时特殊处理，避免逻辑矛盾。而二级缓存缺乏这种动态决策能力，因此无法替代三级缓存。

## spring 三级缓存的数据结构是什么?

都是 Map 类型的缓存，比如 Map {k:name; v:bean}。

1. **一级缓存（Singleton Objects）**：这是一个 Map 类型的缓存，存储的是已经完全初始化好的 bean，即完全准备好可以使用的 bean 实例。键是 bean 的名称，值是 bean 的实例。这个缓存在 `DefaultSingletonBeanRegistry` 类中的 `singletonObjects` 属性中。
2. **二级缓存（Early Singleton Objects）**：这同样是一个 Map 类型的缓存，存储的是早期的 bean 引用，即已经实例化但还未完全初始化的 bean。这些 bean 已经被实例化，但是可能还没有进行属性注入等操作。这个缓存在 `DefaultSingletonBeanRegistry` 类中的 `earlySingletonObjects` 属性中。
3. **三级缓存（Singleton Factories）**：这也是一个 Map 类型的缓存，存储的是 ObjectFactory 对象，这些对象可以生成早期的 bean 引用。当一个 bean 正在创建过程中，如果它被其他 bean 依赖，那么这个正在创建的 bean 就会通过这个 ObjectFactory 来创建一个早期引用，从而解决循环依赖的问题。这个缓存在 `DefaultSingletonBeanRegistry` 类中的 `singletonFactories` 属性中。

## spring 框架中都用到了哪些设计模式

- **工厂设计模式** ：Spring 使用工厂模式通过 BeanFactory、ApplicationContext 创建 bean 对象。
- **代理设计模式** ：Spring AOP 功能的实现。
- **单例设计模式** ：Spring 中的 Bean 默认都是单例的。
- **模板方法模式** ：Spring 中 jdbcTemplate、hibernateTemplate 等以 Template 结尾的对数据库操作的类，它们就使用到了模板模式。
- **包装器设计模式** ：我们的项目需要连接多个数据库，而且不同的客户在每次访问中根据需要会去访问不同的数据库。这种模式让我们可以根据客户的需求能够动态切换不同的数据源。
- **观察者模式**: Spring 事件驱动模型就是观察者模式很经典的一个应用。
- **适配器模式** :Spring AOP 的增强或通知(Advice)使用到了适配器模式、spring MVC 中也是用到了适配器模式适配 Controller。

## spring 常用注解有什么?

> @Autowired 注解

@Autowired：主要用于自动装配 bean。当 Spring 容器中存在与要注入的属性类型匹配的 bean 时，它会自动将 bean 注入到属性中。就跟我们 new 对象一样。

用法很简单，如下示例代码：

```
@Component
public class MyService {
}

@Component
public class MyController {

    @Autowired
    private MyService myService;

}
```

在上面的示例代码中，MyController 类中的 myService 属性被@Autowired 注解标记，Spring 会自动将 MyService 类型的 bean 注入到 myService 属性中。

> @Component

这个注解用于标记一个类作为 Spring 的 bean。当一个类被@Component 注解标记时，Spring 会将其实例化为一个 bean，并将其添加到 Spring 容器中。在上面讲解@Autowired 的时候也看到了，示例代码：

```
@Component
public class MyComponent {
}
```

在上面的示例代码中，MyComponent 类被@Component 注解标记，Spring 会将其实例化为一个 bean，并将其添加到 Spring 容器中。

> @Configuration

@Configuration，注解用于标记一个类作为 Spring 的配置类。配置类可以包含@Bean 注解的方法，用于定义和配置 bean，作为全局配置。示例代码：

```
@Configuration
public class MyConfiguration {

    @Bean
    public MyBean myBean() {
```

```
        return new MyBean();
    }

}
```

> @Bean

@Bean 注解用于标记一个方法作为 Spring 的 bean 工厂方法。当一个方法被@Bean 注解标记时，Spring 会将该方法的返回值作为一个 bean，并将其添加到 Spring 容器中，如果自定义配置，经常用到这个注解。

```
@Configuration
public class MyConfiguration {

    @Bean
    public MyBean myBean() {
        return new MyBean();

    }

}
```

> @Service

@Service，这个注解用于标记一个类作为服务层的组件。它是@Component 注解的特例，用于标记服务层的 bean，一般标记在业务 service 的实现类。

```
@Service
public class MyServiceImpl {

}
```

> @Repository

@Repository 注解用于标记一个类作为数据访问层的组件。它也是@Component 注解的特例，用于标记数据访问层的 bean。这个注解很容易被忽略，导致数据库无法访问。

```
@Repository
```

```
public class MyRepository {


}
```

在上面的示例代码中，MyRepository 类被@Repository 注解标记，Spring 会将其实例化为一个 bean，并将其添加到 Spring 容器中。

> @Controller

@Controller 注解用于标记一个类作为控制层的组件。它也是@Component 注解的特例，用于标记控制层的 bean。这是 MVC 结构的另一个部分，加在控制层

```
@Controller
public class MyController {


}
```

在上面的示例代码中，MyController 类被@Controller 注解标记，Spring 会将其实例化为一个 bean，并将其添加到 Spring 容器中。

## Spring 的事务什么情况下会失效?

Spring Boot 通过 Spring 框架的事务管理模块来支持事务操作。事务管理在 Spring Boot 中通常是通过 @Transactional 注解来实现的。事务可能会失效的一些常见情况包括:

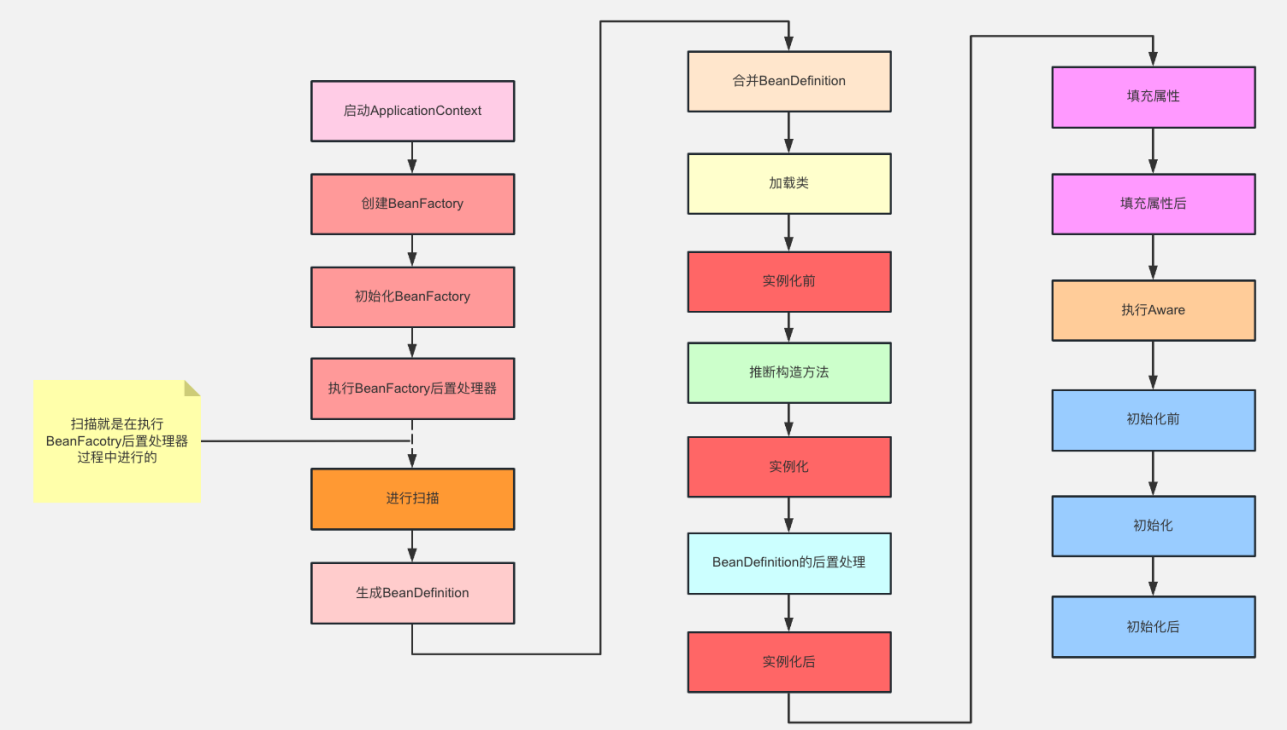
1. **未捕获异常**: 如果一个事务方法中发生了未捕获的异常，并且异常未被处理或传播到事务边界之外，那么事务会失效，所有的数据库操作会回滚。
2. **非受检异常**: 默认情况下，Spring 对非受检异常（RuntimeException 或其子类）进行回滚处理，这意味着当事务方法中抛出这些异常时，事务会回滚。
3. **事务传播属性设置不当**: 如果在多个事务之间存在事务嵌套，且事务传播属性配置不正确，可能导致事务失效。特别是在方法内部调用有 @Transactional 注解的方法时要特别注意。
4. **多数据源的事务管理**: 如果在使用多数据源时，事务管理没有正确配置或者存在多个 @Transactional 注解时，可能会导致事务失效。
5. **跨方法调用事务问题**: 如果一个事务方法内部调用另一个方法，而这个被调用的方法没有 @Transactional 注解，这种情况下外层事务可能会失效。
6. **事务在非公开方法中失效**: 如果 @Transactional 注解标注在私有方法上或者非 public 方法上，事务也会失效。

## Spring 的事务，使用 this 调用是否生效?

不能生效。

因为 Spring 事务是通过代理对象来控制的，只有通过代理对象的方法调用才会应用事务管理的相关规则。当使用 `this` 直接调用时，是绕过了 Spring 的代理机制，因此不会应用事务设置。

## Bean 的生命周期说一下?




1. Spring 启动，查找并加载需要被 Spring 管理的 bean，进行 Bean 的实例化
2. Bean 实例化后对将 Bean 的引入和值注入到 Bean 的属性中
3. 如果 Bean 实现了 BeanNameAware 接口的话，Spring 将 Bean 的 Id 传递给 setBeanName()方法
4. 如果 Bean 实现了 BeanFactoryAware 接口的话，Spring 将调用 setBeanFactory()方法，将 BeanFactory 容器实例传入
5. 如果 Bean 实现了 ApplicationContextAware 接口的话，Spring 将调用 Bean 的 setApplicationContext()方法，将 bean 所在应用上下文引用传入进来。
6. 如果 Bean 实现了 BeanPostProcessor 接口，Spring 就将调用他们的 postProcessBeforeInitialization()方法。
7. 如果 Bean 实现了 InitializingBean 接口，Spring 将调用他们的 afterPropertiesSet()方法。类似的，如果 bean 使用 init-method 声明了初始化方法，该方法也会被调用
8. 如果 Bean 实现了 BeanPostProcessor 接口，Spring 就将调用他们的 postProcessAfterInitialization()方法。

9. 此时，Bean 已经准备就绪，可以被应用程序使用了。他们将一直驻留在应用上下文中，直到应用上下文被销毁。
10. 如果 bean 实现了 DisposableBean 接口，Spring 将调用它的 destory()接口方法，同样，如果 bean 使用了 destory-method 声明销毁方法，该方法也会被调用。

## Bean 是否单例?

Spring 中的 Bean 默认都是单例的。

就是说，每个 Bean 的实例只会被创建一次，并且会被存储在 Spring 容器的缓存中，以便在后续的请求中重复使用。这种单例模式可以提高应用程序的性能和内存效率。

但是，Spring 也支持将 Bean 设置为多例模式，即每次请求都会创建一个新的 Bean 实例。要将 Bean 设置为多例模式，可以在 Bean 定义中通过设置 scope 属性为"prototype"来实现。

需要注意的是，虽然 Spring 的默认行为是将 Bean 设置为单例模式，但在一些情况下，使用多例模式是更为合适的，例如在创建状态不可变的 Bean 或有状态 Bean 时。此外，需要注意的是，如果 Bean 单例是有状态的，那么在使用时需要考虑线程安全性问题。

## Bean 的单例和非单例，生命周期是否一样

不一样的，Spring Bean 的生命周期完全由 IoC 容器控制。Spring 只帮我们管理单例模式 Bean 的完整生命周期，对于 `prototype` 的 Bean，Spring 在创建好交给使用者之后，则不会再管理后续的生命周期。

具体区别如下：

| 阶段 | 单例（Singleton） | 非单例（如 Prototype） |
| --- | --- | --- |
| **创建时机** | 容器启动时创建（或首次请求时，取决于配置）。 | 每次请求时创建新实例。 |
| **初始化流程** | 完整执行生命周期流程（属性注入、Aware 接口、初始化方法等）。 | 每次创建新实例时都会完整执行生命周期流程（仅到初始化完成）。 |

| 阶段 | 单例（Singleton） | 非单例（如 Prototype） |
| --- | --- | --- |
| **销毁时机** | 容器关闭时销毁，触发 `DisposableBean` 或 `destroy-method`。 | **容器不管理销毁**，需由调用者自行释放资源（Spring 不跟踪实例）。 |
| **内存占用** | 单实例常驻内存，高效但需注意线程安全。 | 每次请求生成新实例，内存开销较大，需手动管理资源释放。 |
| **适用场景** | 无状态服务（如 Service、DAO 层）。 | 有状态对象（如用户会话、临时计算对象）。 |

## Spring bean 的作用域有哪些?

Spring 框架中的 Bean 作用域（Scope）定义了 Bean 的生命周期和可见性。不同的作用域影响着 Spring 容器如何管理这些 Bean 的实例，包括它们如何被创建、如何被销毁以及它们是否可以被多个用户共享。

Spring 支持几种不同的作用域，以满足不同的应用场景需求。以下是一些主要的 Bean 作用域：

- **Singleton（单例）**：在整个应用程序中只存在一个 Bean 实例。默认作用域，Spring 容器中只会创建一个 Bean 实例，并在容器的整个生命周期中共享该实例。
- **Prototype（原型）**：每次请求时都会创建一个新的 Bean 实例。次从容器中获取该 Bean 时都会创建一个新实例，适用于状态非常瞬时的 Bean。
- **Request（请求）**：每个 HTTP 请求都会创建一个新的 Bean 实例。仅在 Spring Web 应用程序中有效，每个 HTTP 请求都会创建一个新的 Bean 实例，适用于 Web 应用中需求局部性的 Bean。
- **Session（会话）**：Session 范围内只会创建一个 Bean 实例。该 Bean 实例在用户会话范围内共享，仅在 Spring Web 应用程序中有效，适用于与用户会话相关的 Bean。
- **Application**：当前 ServletContext 中只存在一个 Bean 实例。仅在 Spring Web 应用程序中有效，该 Bean 实例在整个 ServletContext 范围内共享，适用于应用程序范围内共享的 Bean。

- **WebSocket（Web 套接字）**：在 WebSocket 范围内只存在一个 Bean 实例。仅在支持 WebSocket 的应用程序中有效，该 Bean 实例在 WebSocket 会话范围内共享，适用于 WebSocket 会话范围内共享的 Bean。
- **Custom scopes（自定义作用域）**：Spring 允许开发者定义自定义的作用域，通过实现 Scope 接口来创建新的 Bean 作用域。

在 Spring 配置文件中，可以通过`<bean>`标签的 scope 属性来指定 Bean 的作用域。例如：

```
<bean id="myBean" class="com.example.MyBeanClass" scope="singleton"/>
```

在 Spring Boot 或基于 Java 的配置中，可以通过@Scope 注解来指定 Bean 的作用域。例如：

```
@Bean
@Scope("prototype")
public MyBeanClass myBean() {
    return new MyBeanClass();
}
```

## Spring 容器里存的是什么?

在 Spring 容器中，存储的**主要是 Bean 对象**。

Bean 是 Spring 框架中的基本组件，用于表示应用程序中的各种对象。当应用程序启动时，Spring 容器会根据配置文件或注解的方式创建和管理这些 Bean 对象。Spring 容器会负责创建、初始化、注入依赖以及销毁 Bean 对象。

## 在 Spring 中，在 bean 加载/销毁前后，如果想实现某些逻辑，可以怎么做

在 Spring 框架中，如果你希望在 Bean 加载（即实例化、属性赋值、初始化等过程完成后）或销毁前后执行某些逻辑，你可以使用 Spring 的生命周期回调接口或注解。这些接口和注解允许你定义在 Bean 生命周期的关键点执行的代码。

> 使用 init-method 和 destroy-method

在 XML 配置中，你可以通过 init-method 和 destroy-method 属性来指定 Bean 初始化后和销毁前需要调用的方法。

```
<bean id="myBean" class="com.example.MyBeanClass"
      init-method="init" destroy-method="destroy"/>
```

然后，在你的 Bean 类中实现这些方法：

```
public class MyBeanClass {

    public void init() {
        // 初始化逻辑
    }

    public void destroy() {
        // 销毁逻辑
    }
}
```

> 实现 InitializingBean 和 DisposableBean 接口

你的 Bean 类可以实现 org.springframework.beans.factory.InitializingBean 和 org.springframework.beans.factory.DisposableBean 接口，并分别实现 afterPropertiesSet 和 destroy 方法。

```
import org.springframework.beans.factory.DisposableBean;
import org.springframework.beans.factory.InitializingBean;

public class MyBeanClass implements InitializingBean, DisposableBean {

    @Override
    public void afterPropertiesSet() throws Exception {
        // 初始化逻辑
    }

    @Override
    public void destroy() throws Exception {
        // 销毁逻辑
    }
}
```

> 使用@PostConstruct 和@PreDestroy 注解

```
import javax.annotation.PostConstruct;
import javax.annotation.PreDestroy;

public class MyBeanClass {

    @PostConstruct
    public void init() {
        // 初始化逻辑
    }

    @PreDestroy
    public void destroy() {
        // 销毁逻辑
    }
}
```

> 使用@Bean 注解的 initMethod 和 destroyMethod 属性

在基于 Java 的配置中，你还可以在@Bean 注解中指定 initMethod 和 destroyMethod 属性。

```
@Configuration
public class AppConfig {

    @Bean(initMethod = "init", destroyMethod = "destroy")
    public MyBeanClass myBean() {
        return new MyBeanClass();
    }
}
```

## Spring 给我们提供了很多扩展点，这些有了解吗?

Spring 框架提供了许多扩展点，使得开发者可以根据需求定制和扩展 Spring 的功能。以下是一些常用的扩展点：

1. BeanFactoryPostProcessor：允许在 Spring 容器实例化 bean 之前修改 bean 的定义。常用于修改 bean 属性或改变 bean 的作用域。

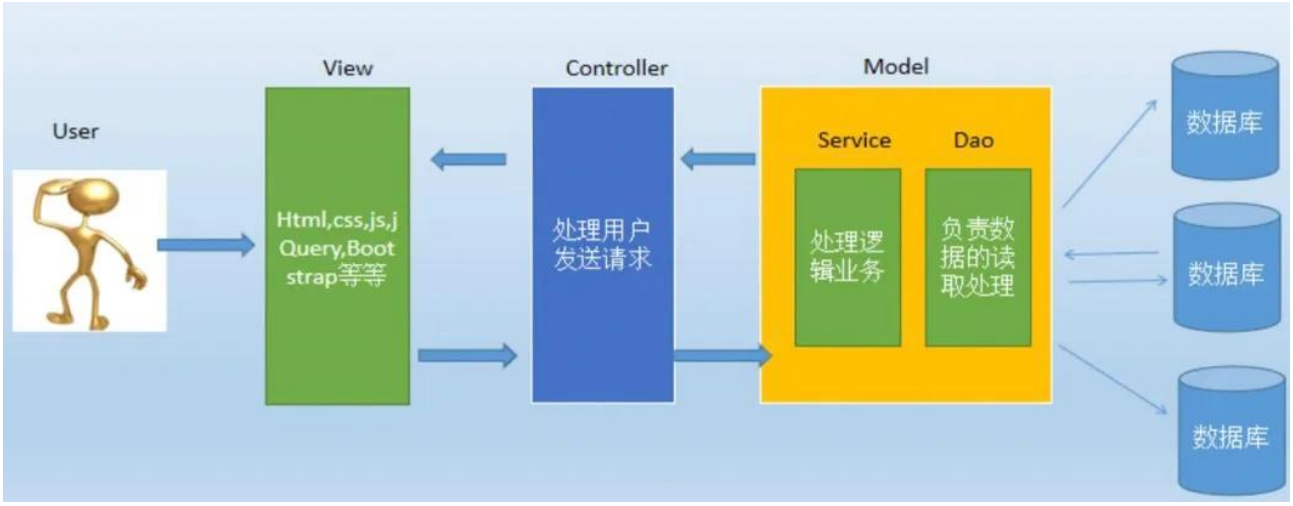
2. BeanPostProcessor：可以在 bean 实例化、配置以及初始化之后对其进行额外处理。常用于代理 bean、修改 bean 属性等。
3. PropertySource：用于定义不同的属性源，如文件、数据库等，以便在 Spring 应用中使用。
4. ImportSelector 和 ImportBeanDefinitionRegistrar：用于根据条件动态注册 bean 定义，实现配置类的模块化。
5. Spring MVC 中的 HandlerInterceptor：用于拦截处理请求，可以在请求处理前、处理中和处理后执行特定逻辑。
6. Spring MVC 中的 ControllerAdvice：用于全局处理控制器的异常、数据绑定和数据校验。
7. Spring Boot 的自动配置：通过创建自定义的自动配置类，可以实现对框架和第三方库的自动配置。
8. 自定义注解：创建自定义注解，用于实现特定功能或约定，如权限控制、日志记录等。

# SpringMVC

## MVC 分层介绍一下

MVC 全名是 Model View Controller，是模型(model)－视图(view)－控制器(controller)的缩写，一种软件设计典范，用一种业务逻辑、数据、界面显示分离的方法组织代码，将业务逻辑聚集到一个部件里面，在改进和个性化定制界面及用户交互的同时，不需要重新编写业务逻辑。

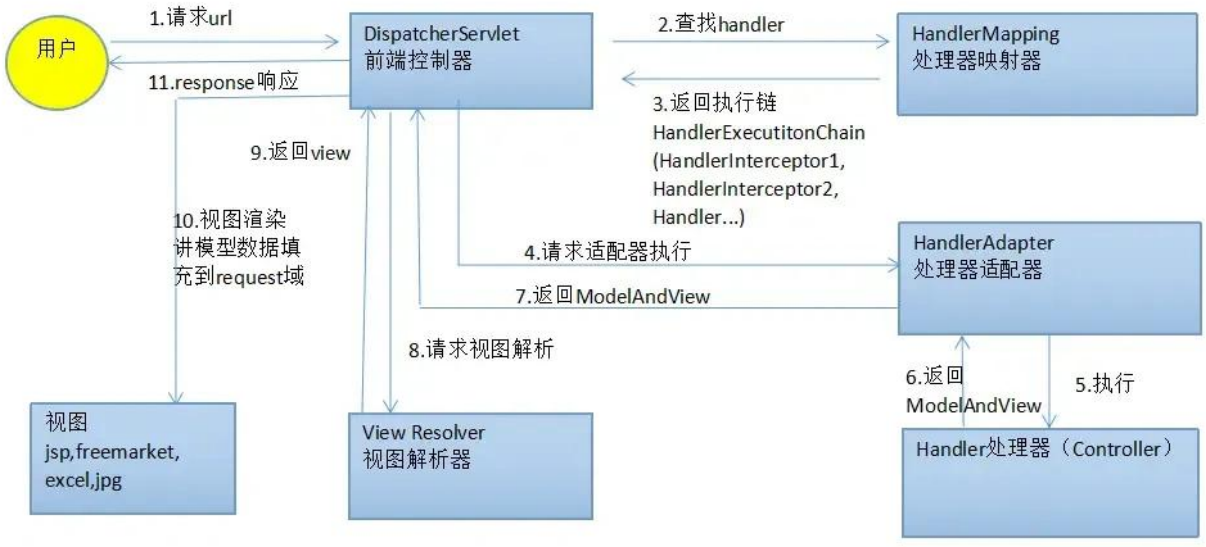
- **视图(view)： 为用户提供使用界面，与用户直接进行交互。**
- **模型(model)： 代表一个存取数据的对象或 JAVA POJO（Plain Old Java Object，简单 java 对象）。它也可以带有逻辑，主要用于承载数据，并对用户提交请求进行计算的模块。模型分为两类，一类称为数据承载 Bean，一类称为业务处理 Bean。所谓数据承载 Bean 是指实体类（如：User 类），专门为用户承载业务数据的；而业务处理 Bean 则是指 Service 或 Dao 对象， 专门用于处理用户提交请求的。**
- **控制器(controller)： 用于将用户请求转发给相应的 Model 进行处理，并根据 Model 的计算结果向用户提供相应响应。它使视图与模型分离。**




流程步骤：

1. 用户通过 View 页面向服务端提出请求，可以是表单请求、超链接请求、AJAX 请求等；
2. 服务端 Controller 控制器接收到请求后对请求进行解析，找到相应的 Model，对用户请求进行处理 Model 处理；
3. 将处理结果再交给 Controller（控制器其实只是起到了承上启下的作用）；
4. 根据处理结果找到要作为向客户端发回的响应 View 页面，页面经渲染后发送给客户端。

## 了解 SpringMVC 的处理流程吗?




Spring MVC 的工作流程如下：

1. 用户发送请求至前端控制器 DispatcherServlet
2. DispatcherServlet 收到请求调用处理器映射器 HandlerMapping。

3. 处理器映射器根据请求 url 找到具体的处理器，生成处理器执行链 HandlerExecutionChain(包括处理器对象和处理器拦截器)一并返回给 DispatcherServlet。
4. DispatcherServlet 根据处理器 Handler 获取处理器适配器 HandlerAdapter 执行 HandlerAdapter 处理一系列的操作，如：参数封装，数据格式转换，数据验证等操作
5. 执行处理器 Handler(Controller，也叫页面控制器)。
6. Handler 执行完成返回 ModelAndView
7. HandlerAdapter 将 Handler 执行结果 ModelAndView 返回到 DispatcherServlet
8. DispatcherServlet 将 ModelAndView 传给 ViewReslover 视图解析器
9. ViewReslover 解析后返回具体 View
10. DispatcherServlet 对 View 进行渲染视图（即将模型数据 model 填充至视图中）。
11. DispatcherServlet 响应用户。

## Handlermapping 和 handleradapter 有了解吗?

HandlerMapping：

- **作用**：HandlerMapping 负责将请求映射到处理器（Controller）。
- **功能**：根据请求的 URL、请求参数等信息，找到处理请求的 Controller。
- **类型**：Spring 提供了多种 HandlerMapping 实现，如 BeanNameUrlHandlerMapping、RequestMappingHandlerMapping 等。
- **工作流程**：根据请求信息确定要请求的处理器(Controller)。HandlerMapping 可以根据 URL、请求参数等规则确定对应的处理器。

HandlerAdapter：

- **作用**：HandlerAdapter 负责调用处理器(Controller)来处理请求。
- **功能**：处理器(Controller)可能有不同的接口类型（Controller 接口、HttpRequestHandler 接口等），HandlerAdapter 根据处理器的类型来选择合适的方法来调用处理器。
- **类型**：Spring 提供了多个 HandlerAdapter 实现，用于适配不同类型的处理器。
- **工作流程**：根据处理器的接口类型，选择相应的 HandlerAdapter 来调用处理器。

工作流程：

1. 当客户端发送请求时，HandlerMapping 根据请求信息找到对应的处理器(Controller)。
2. HandlerAdapter 根据处理器的类型选择合适的方法来调用处理器。

3. 处理器执行相应的业务逻辑，生成 ModelAndView。
4. HandlerAdapter 将处理器的执行结果包装成 ModelAndView。
5. 视图解析器根据 ModelAndView 找到对应的视图进行渲染。
6. 将渲染后的视图返回给客户端。

HandlerMapping 和 HandlerAdapter 协同工作，通过将请求映射到处理器，并调用处理器来处理请求，实现了请求处理的流程。它们的灵活性使得在 Spring MVC 中可以支持多种处理器和处理方式，提高了框架的扩展性和适应性。

# SpringBoot

## 为什么使用 springboot

- 简化开发：Spring Boot 通过提供一系列的开箱即用的组件和自动配置，简化了项目的配置和开发过程，开发人员可以更专注于业务逻辑的实现，而不需要花费过多时间在繁琐的配置上。
- 快速启动：Spring Boot 提供了快速的应用程序启动方式，可通过内嵌的 Tomcat、Jetty 或 Undertow 等容器快速启动应用程序，无需额外的部署步骤，方便快捷。
- 自动化配置：Spring Boot 通过自动配置功能，根据项目中的依赖关系和约定俗成的规则来配置应用程序，减少了配置的复杂性，使开发者更容易实现应用的最佳实践。

## SpringBoot 比 Spring 好在哪里

- Spring Boot 提供了自动化配置，大大简化了项目的配置过程。通过约定优于配置的原则，很多常用的配置可以自动完成，开发者可以专注于业务逻辑的实现。
- Spring Boot 提供了快速的项目启动器，通过引入不同的 Starter，可以快速集成常用的框架和库（如数据库、消息队列、Web 开发等），极大地提高了开发效率。
- Spring Boot 默认集成了多种内嵌服务器（如 Tomcat、Jetty、Undertow），无需额外配置，即可将应用打包成可执行的 JAR 文件，方便部署和运行。

## SpringBoot 用到哪些设计模式?

- **代理模式**：Spring 的 AOP 通过动态代理实现方法级别的切面增强，有静态和动态两种代理方式，采用动态代理方式。
- **策略模式**：Spring AOP 支持 JDK 和 Cglib 两种动态代理实现方式，通过策略接口和不同策略类，运行时动态选择，其创建一般通过工厂方法实现。

- **装饰器模式**：Spring 用 TransactionAwareCacheDecorator 解决缓存与数据库事务问题增加对事务的支持。
- **单例模式**：Spring Bean 默认是单例模式，通过单例注册表（如 HashMap）实现。
- **简单工厂模式**：Spring 中的 BeanFactory 是简单工厂模式的体现，通过工厂类方法获取 Bean 实例。
- **工厂方法模式**：Spring 中的 FactoryBean 体现工厂方法模式，为不同产品提供不同工厂。
- **观察者模式**：Spring 观察者模式包含 Event 事件、Listener 监听者、Publisher 发送者，通过定义事件、监听器和发送者实现，观察者注册在 ApplicationContext 中，消息发送由 ApplicationEventMulticaster 完成。
- **模板模式**：Spring Bean 的创建过程涉及模板模式，体现扩展性，类似 Callback 回调实现方式。
- **适配器模式**：Spring MVC 中针对不同方式定义的 Controller，利用适配器模式统一函数定义，定义了统一接口 HandlerAdapter 及对应适配器类。

## 怎么理解 SpringBoot 中的约定大于配置

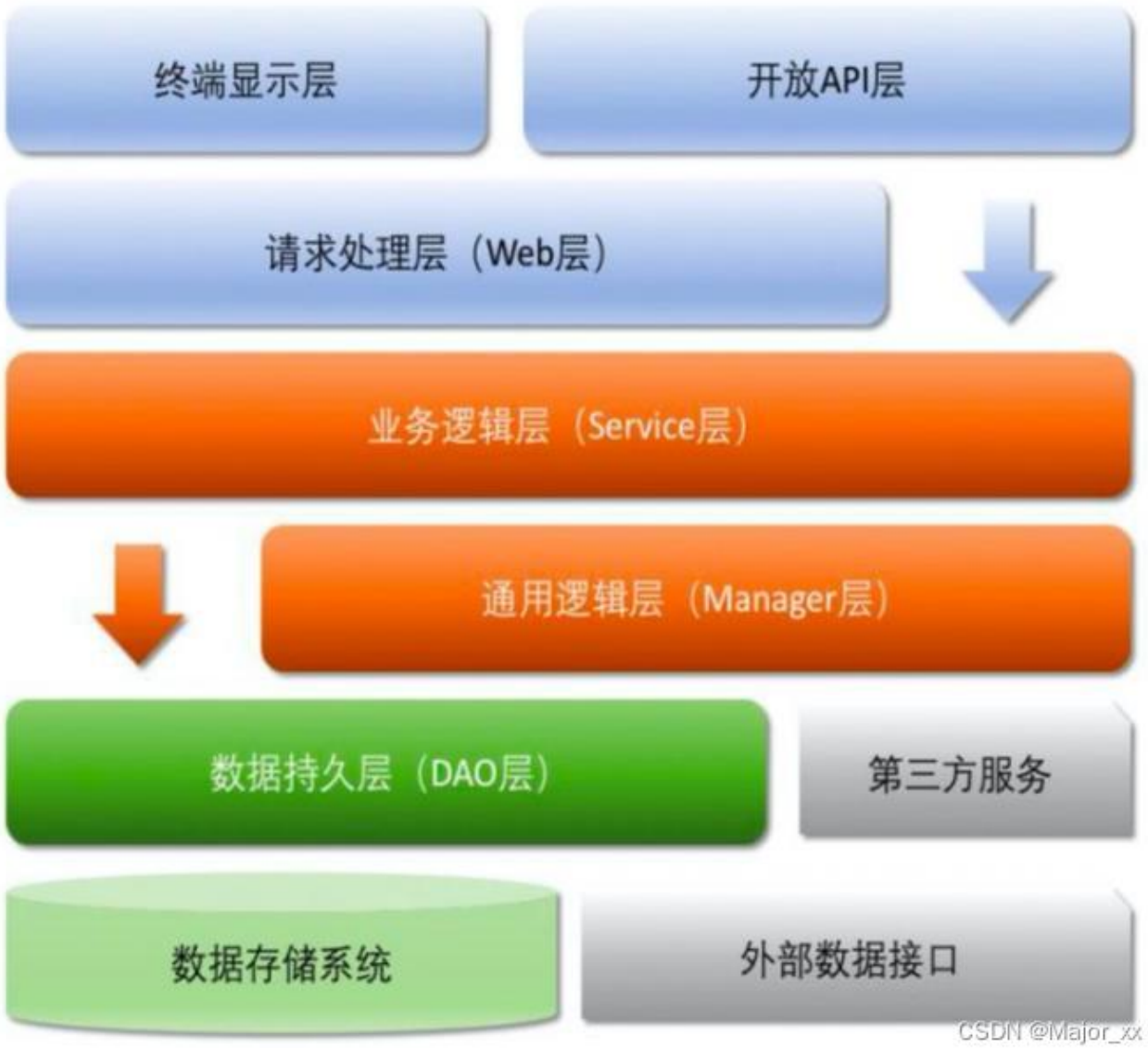
约定大于配置是 Spring Boot 的核心设计理念，它通过**预设合理的默认行为和项目规范**，大幅减少开发者需要手动配置的步骤，从而提升开发效率和项目标准化程度。

理解 Spring Boot 中的“约定大于配置”原则，可以从以下几个方面来解释：

- **自动化配置**：Spring Boot 提供了大量的自动化配置，通过分析项目的依赖和环境，自动配置应用程序的行为。开发者无需显式地配置每个细节，大部分常用的配置都已经预设好了。例如，引入 `spring-boot-starter-web` 后，Spring Boot 会自动配置内嵌 Tomcat 和 Spring MVC，无需手动编写 XML。
- **默认配置**：Spring Boot 为诸多方面提供大量默认配置，如连接数据库、设置 Web 服务器、处理日志等。开发人员无需手动配置这些常见内容，框架已做好决策。例如，默认的日志配置可让应用程序快速输出日志信息，无需开发者额外繁琐配置日志级别、输出格式与位置等。
- **约定的项目结构**：Spring Boot 提倡特定项目结构，通常主应用程序类（含 main 方法）置于根包，控制器类、服务类、数据访问类等分别放在相应子包，如 `com.example.demo.controller` 放控制器类，`com.example.demo.service` 放服务类等。此约定使团队成员更易理解项目结构与组织，新成员加入项目时能快速定位各功能代码位置，提升协作效率。

## SpringBoot 的项目结构是怎么样的?

一个正常的企业项目里一种通用的项目结构和代码层级划分的指导意见。按这《阿里巴巴 Java 开发手册》时本书上说的，一般分为如下几层：

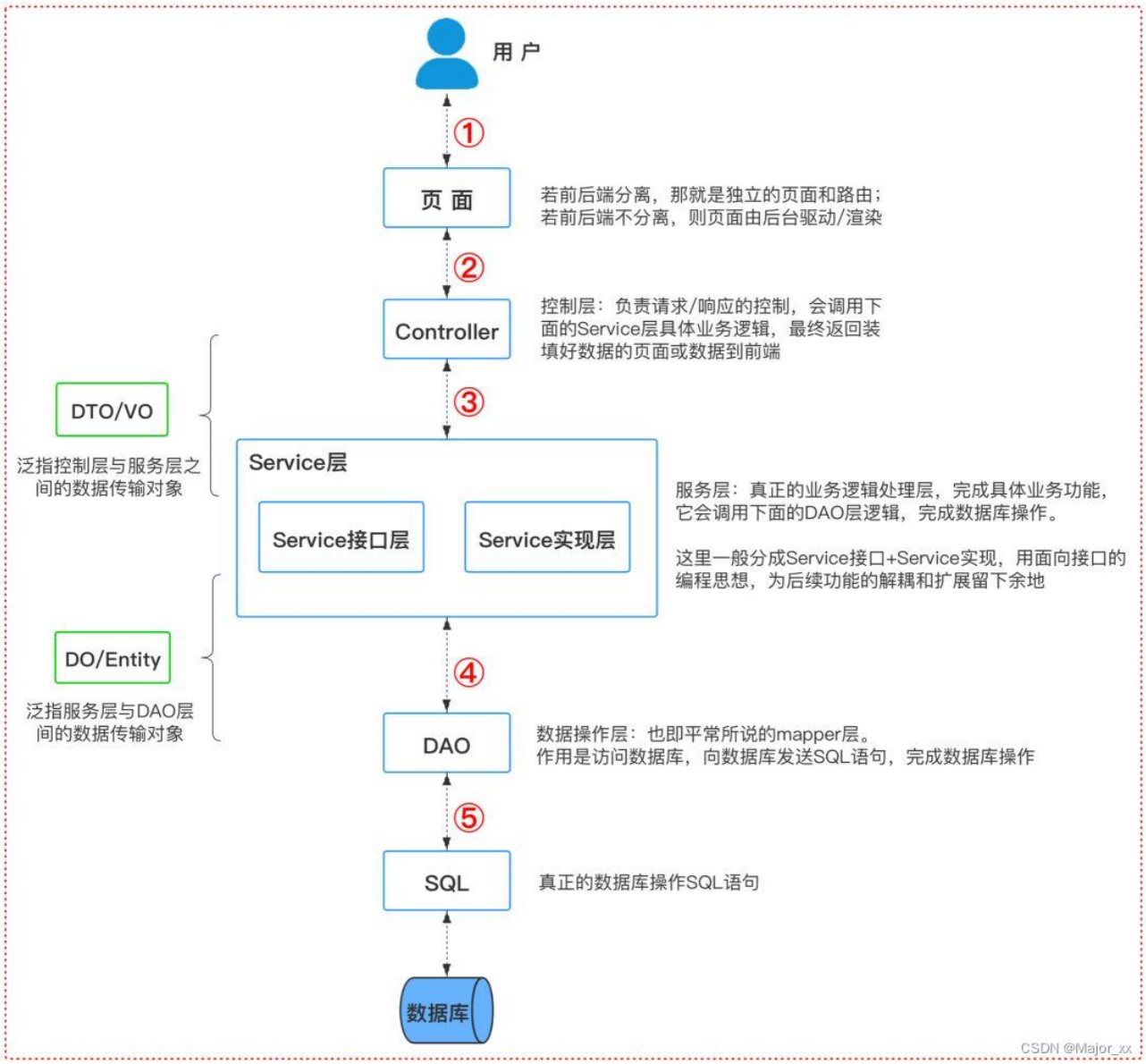



- 开放接口层：可直接封装 Service 接口暴露成 RPC 接口；通过 Web 封装成 http 接口；网关控制层等。
- 终端显示层：各个端的模板渲染并执行显示的层。当前主要是 velocity 渲染，JS 渲染，JSP 渲染，移动端展示等。
- Web 层：主要是对访问控制进行转发，各类基本参数校验，或者不复用的业务简单处理等。
- Service 层：相对具体的业务逻辑服务层。
- Manager 层：通用业务处理层，它有如下特征：
    - 1）对第三方平台封装的层，预处理返回结果及转化异常信息，适配上层接口。

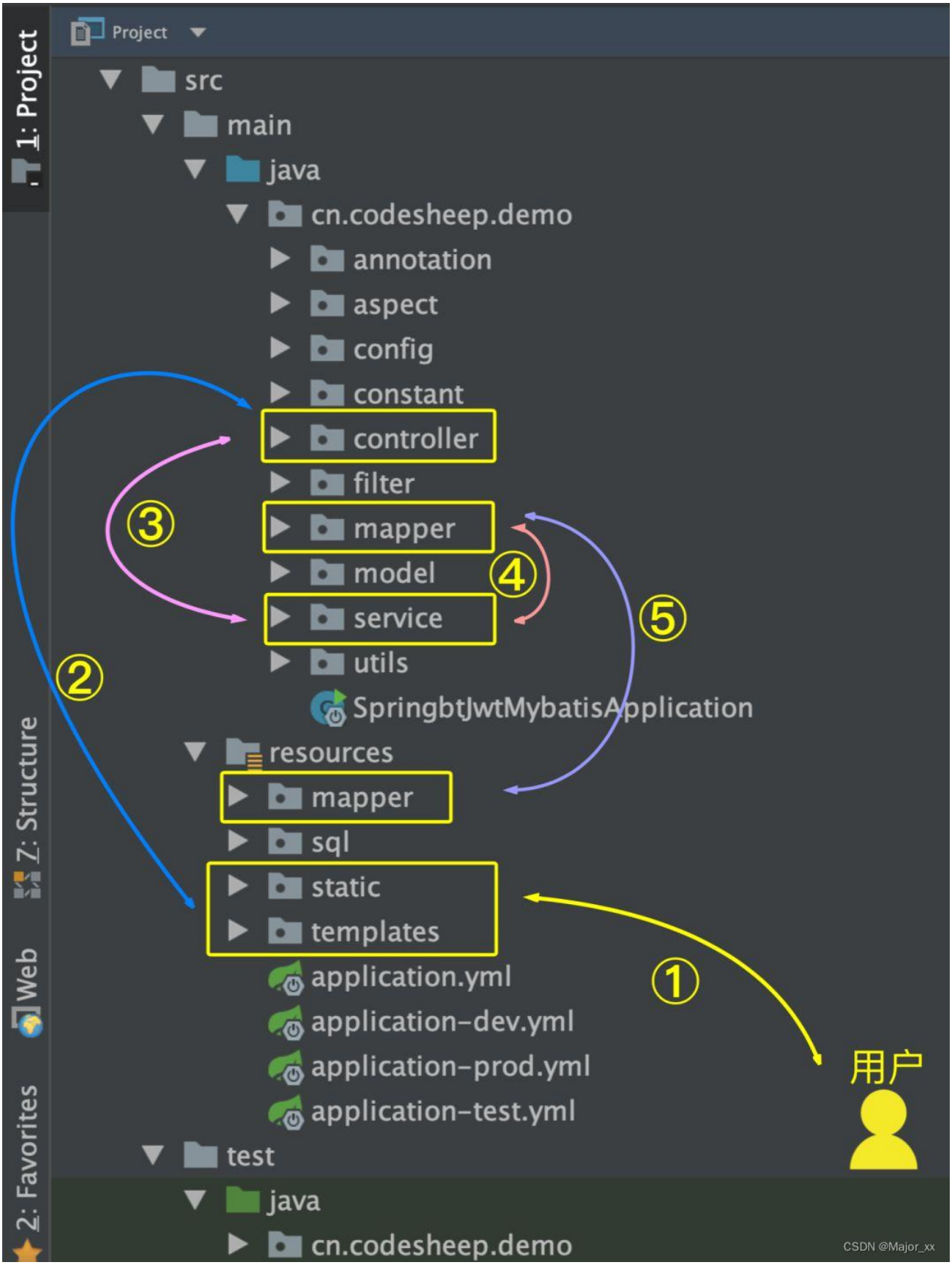
- 2）对 Service 层通用能力的下沉，如缓存方案、中间件通用处理。
- 3）与 DAO 层交互，对多个 DAO 的组合复用。

- DAO 层：数据访问层，与底层 MySQL、Oracle、Hbase、OceanBase 等进行数据交互。
- 第三方服务：包括其它部门 RPC 服务接口，基础平台，其它公司的 HTTP 接口，如淘宝开放平台、支付宝付款服务、高德地图服务等。
- 外部接口：外部（应用）数据存储服务提供的接口，多见于数据迁移场景中。

如果从一个用户访问一个网站的情况来看，对应着上面的项目代码结构来分析，可以贯穿整个代码分层：




对应代码目录的流转逻辑就是：




所以，以后每当我们拿到一个新的项目到手时，只要按照这个思路去看别人项目的代码，应该基本都是能理得顺的。

## SpringBoot 自动装配原理是什么?

> 什么是自动装配?

SpringBoot 的自动装配原理是基于 Spring Framework 的条件化配置和@EnableAutoConfiguration 注解实现的。这种机制允许开发者在项目中引入相关的依赖，SpringBoot 将根据这些依赖自动配置应用程序的上下文和功能。

SpringBoot 定义了一套接口规范，这套规范规定：SpringBoot 在启动时会扫描外部引用 jar 包中的 META-INF/spring.factories 文件，将文件中配置的类型信息加载到 Spring 容器（此处涉及到 JVM 类加载机制与 Spring 的容器知识），并执行类中定义的各种操作。对于外部 jar 来说，只需要按照 SpringBoot 定义的标准，就能将自己的功能装置进 SpringBoot。

通俗来讲，自动装配就是通过注解或一些简单的配置就可以在 SpringBoot 的帮助下开启和配置各种功能，比如数据库访问、Web 开发。

> SpringBoot 自动装配原理

首先点进 `@SpringBootApplication` 注解的内部

```
6 usages
@Target({ElementType.TYPE})
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Inherited
@SpringBootConfiguration
@EnableAutoConfiguration
@ComponentScan(
        excludeFilters = {@Filter(
                type = FilterType.CUSTOM,
                classes = {TypeExcludeFilter.class}
        ), @Filter(
                type = FilterType.CUSTOM,
                classes = {AutoConfigurationExcludeFilter.class}
        )}
)
```

接下来将逐个解释这些注解的作用：

- `@Target({ElementType.TYPE})`：该注解指定了这个注解可以用来标记在类上。在这个特定的例子中，这表示该注解用于标记配置类。
- `@Retention(RetentionPolicy.RUNTIME)`：这个注解指定了注解的生命周期，即在运行时保留。这是因为 Spring Boot 在运行时扫描类路径上的注解来实现自动配置，所以这里使用了 RUNTIME 保留策略。
- `@Documented`：该注解表示这个注解应该被包含在 Java 文档中。它是用于生成文档的标记，使开发者能够看到这个注解的相关信息。

- `@Inherited`：这个注解指示一个被标注的类型是被继承的。在这个例子中，它表明这个注解可以被继承，如果一个类继承了带有这个注解的类，它也会继承这个注解。
- `@SpringBootConfiguration`：这个注解表明这是一个 Spring Boot 配置类。如果点进这个注解内部会发现与标准的 @Configuration 没啥区别，只是为了表明这是一个专门用于 SpringBoot 的配置。
- `@EnableAutoConfiguration`：这个注解是 Spring Boot 自动装配的核心。它告诉 Spring oot 启用自动配置机制，根据项目的依赖和配置自动配置应用程序的上下文。通过这个注解，SpringBoot 将尝试根据类路径上的依赖自动配置应用程序。
- `@ComponentScan`：这个注解用于配置组件扫描的规则。在这里，它告诉 SpringBoot 在指定的包及其子包中查找组件，这些组件包括被注解的类、@Component 注解的类等。其中的 excludeFilters 参数用于指定排除哪些组件，这里使用了两个自定义的过滤器，分别是 TypeExcludeFilter 和 AutoConfigurationExcludeFilter。

`@EnableAutoConfiguration` 这个注解是实现自动装配的核心注解

```
@Target({ElementType.TYPE})
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Inherited
@AutoConfigurationPackage
@Import({AutoConfigurationImportSelector.class})
public @interface EnableAutoConfiguration {
    no usages
    String ENABLED_OVERRIDE_PROPERTY = "spring.boot.enableautoconfiguration";

    Class<?>[] exclude() default {};

    no usages
    String[] excludeName() default {};
}
```

- @AutoConfigurationPackage，将项目 src 中 main 包下的所有组件注册到容器中，例如标注了 Component 注解的类等
- @Import({AutoConfigurationImportSelector.class})，是自动装配的核心，接下来分析一下这个注解

AutoConfigurationImportSelector 是 Spring Boot 中一个重要的类，它实现了 ImportSelector 接口，用于实现自动配置的选择和导入。具体来说，它通过分析项目的类路径和条件来决定应该导入哪些自动配置类。

代码太多，选取部分主要功能的代码：

```
public class AutoConfigurationImportSelector implements DeferredImportSelector,
BeanClassLoaderAware,

                ResourceLoaderAware, BeanFactoryAware, EnvironmentAware, Ordered {


    // ... （其他方法和属性）


  // 获取所有符合条件的类的全限定类名，例如 RedisTemplate 的全限定类名
(org.springframework.data.redis.core.RedisTemplate;)，这些类需要被加载到 IoC 容器中。

        @Override

        public String[] selectImports(AnnotationMetadata annotationMetadata) {

                // 扫描类路径上的 META-INF/spring.factories 文件，获取所有实现了
AutoConfiguration 接口的自动配置类

                List<String> configurations = getCandidateConfigurations(annotationMetadata,
attributes);


                // 过滤掉不满足条件的自动配置类，比如一些自动装配类

                configurations = filter(configurations, annotationMetadata, attributes);


                // 排序自动配置类，根据 @AutoConfigureOrder 和
@AutoConfigureAfter/@AutoConfigureBefore 注解指定的顺序

                sort(configurations, annotationMetadata, attributes);


                // 将满足条件的自动配置类的类名数组返回，这些类将被导入到应用程序上下文中

                return StringUtils.toStringArray(configurations);

        }


        // ... （其他方法）

        protected List<String> getCandidateConfigurations(AnnotationMetadata metadata,
AnnotationAttributes attributes) {

                // 获取自动配置类的候选列表，从 META-INF/spring.factories 文件中读取

                // 通过类加载器加载所有候选类

                List<String> configurations =
SpringFactoriesLoader.loadFactoryNames(getSpringFactoriesLoaderFactoryClass(),

                                getBeanClassLoader());


                // 过滤出实现了 AutoConfiguration 接口的自动配置类

                configurations = configurations.stream()
```

```
                                .filter(this::isEnabled)
                                .collect(Collectors.toList());

                // 对于 Spring Boot 1.x 版本，还需要添加 spring-boot-autoconfigure 包中的自动配置类
                //
configurations.addAll(getAutoConfigEntry(getAutoConfigurationEntry(metadata)));
                return configurations;
        }

        // ... （其他方法）
        protected List<String> filter(List<String> configurations, AnnotationMetadata
metadata,
                        AnnotationAttributes attributes) {
                // 使用条件判断机制，过滤掉不满足条件的自动配置类
                configurations = configurations.stream()
                                .filter(configuration ->
isConfigurationCandidate(configuration, metadata, attributes))
                                .collect(Collectors.toList());
                return configurations;
        }

        // ... （其他方法）
        protected void sort(List<String> configurations, AnnotationMetadata metadata,
                        AnnotationAttributes attributes) {
                // 根据 @AutoConfigureOrder 和 @AutoConfigureAfter/@AutoConfigureBefore 注解指定的顺序对自动配置类进行排序
                configurations.sort((o1, o2) -> {
                        int i1 = getAutoConfigurationOrder(o1, metadata, attributes);
                        int i2 = getAutoConfigurationOrder(o2, metadata, attributes);
                        return Integer.compare(i1, i2);
                });
        }

        // ... （其他方法）

}
```

梳理一下，以下是 `AutoConfigurationImportSelector` 的主要工作：

- 扫描类路径：在应用程序启动时，`AutoConfigurationImportSelector` 会扫描类路径上的 `META-INF/spring.factories` 文件，这个文件中包含了各种 Spring 配置和扩展的定义。在这里，它会查找所有实现了 `AutoConfiguration` 接口的类,具体的实现为 `getCandidateConfigurations` 方法。
- 条件判断：对于每一个发现的自动配置类，`AutoConfigurationImportSelector` 会使用条件判断机制（通常是通过 `@ConditionalOnXxx` 注解）来确定是否满足导入条件。这些条件可以是配置属性、类是否存在、Bean 是否存在等等。
- 根据条件导入自动配置类：满足条件的自动配置类将被导入到应用程序的上下文中。这意味着它们会被实例化并应用于应用程序的配置。

## 说几个启动器（starter)?

- spring-boot-starter-web：这是最常用的起步依赖之一，它包含了 Spring MVC 和 Tomcat 嵌入式服务器，用于快速构建 Web 应用程序。
- spring-boot-starter-security：提供了 Spring Security 的基本配置，帮助开发者快速实现应用的安全性，包括认证和授权功能。
- mybatis-spring-boot-starter：这个 Starter 是由 MyBatis 团队提供的，用于简化在 Spring Boot 应用中集成 MyBatis 的过程。它自动配置了 MyBatis 的相关组件，包括 SqlSessionFactory、MapperScannerConfigurer 等，使得开发者能够快速地开始使用 MyBatis 进行数据库操作。
- spring-boot-starter-data-jpa 或 spring-boot-starter-jdbc：如果使用的是 Java Persistence API (JPA)进行数据库操作，那么应该使用 spring-boot-starter-data-jpa。这个 Starter 包含了 Hibernate 等 JPA 实现以及数据库连接池等必要的库，可以让你轻松地与 MySQL 数据库进行交互。你需要在 application.properties 或 application.yml 中配置 MySQL 的连接信息。如果倾向于直接使用 JDBC 而不通过 JPA，那么可以使用 spring-boot-starter-jdbc，它提供了基本的 JDBC 支持。
- spring-boot-starter-data-redis：用于集成 Redis 缓存和数据存储服务。这个 Starter 包含了与 Redis 交互所需的客户端（默认是 Jedis 客户端，也可以配置为 Lettuce 客户端），以及 Spring Data Redis 的支持，使得在 Spring Boot 应用中使用 Redis 变得非常便捷。同样地，需要在配置文件中设置 Redis 服务器的连接详情。

- spring-boot-starter-test：包含了单元测试和集成测试所需的库，如 JUnit, Spring Test, AssertJ 等，便于进行测试驱动开发(TDD)。

## 写过 SpringBoot starter 吗?

> 步骤 1: 创建 Maven 项目

首先，需要创建一个新的 Maven 项目。在 pom.xml 中添加 Spring Boot 的 starter parent 和一些必要的依赖。例如：

```
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>2.7.0</version>
    <relativePath/> <!-- lookup parent from repository -->
</parent>

<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
</dependencies>
```

> 步骤 2: 添加自动配置

在 src/main/resources/META-INF/spring.factories 中添加自动配置的元数据。例如：

```
org.springframework.boot.autoconfigure.EnableAutoConfiguration =
com.example.starter.MyAutoConfiguration
```

然后，创建 MyAutoConfiguration 类，该类需要@Configuration 和 @EnableConfigurationProperties 注解。@EnableConfigurationProperties 用于启用你定义的配置属性类。

```
@Configuration
@EnableConfigurationProperties(MyProperties.class)
public class MyAutoConfiguration {
```

```
    @Autowired
    private MyProperties properties;

    @Bean
    public MyService myService() {
        return new MyServiceImpl(properties);
    }
}
```

> 步骤 3: 创建配置属性类

创建一个配置属性类，使用@ConfigurationProperties 注解来绑定配置文件中的属性。

```
@ConfigurationProperties(prefix = "my")
public class MyProperties {
    private String name;
    // getters and setters
}
```

> 步骤 4: 创建服务和控制器

创建一个服务类和服务实现类，以及一个控制器来展示你的 starter 的功能。

```
@Service
public interface MyService {
    String getName();
}

@Service
public class MyServiceImpl implements MyService {
    private final MyProperties properties;

    public MyServiceImpl(MyProperties properties) {
        this.properties = properties;
    }

    @Override
```

```
    public String getName() {
        return properties.getName();
    }
}

@RestController
public class MyController {
    private final MyService myService;

    public MyController(MyService myService) {
        this.myService = myService;
    }

    @GetMapping("/name")
    public String getName() {
        return myService.getName();
    }
}
```

> 步骤 5: 发布 Starter

将你的 starter 发布到 Maven 仓库，无论是私有的还是公共的，如 Nexus 或 Maven Central。

> 步骤 6: 使用 Starter

在你的主应用的 pom.xml 中添加你的 starter 依赖，然后在 application.yml 或 application.properties 中配置你的属性。

```
my:
    name: Hello World
```

## SpringBoot 里面有哪些重要的注解？还有一个配置相关的注解是哪个？

Spring Boot 中一些常用的注解包括：

- **@SpringBootApplication**：用于标注主应用程序类，标识一个 Spring Boot 应用程序的入口点，同时启用自动配置和组件扫描。
- **@Controller**：标识控制器类，处理 HTTP 请求。

- **@RestController**：结合@Controller 和@ResponseBody，返回 RESTful 风格的数据。
- **@Service**：标识服务类，通常用于标记业务逻辑层。
- **@Repository**：标识数据访问组件，通常用于标记数据访问层。
- **@Component**：通用的 Spring 组件注解，表示一个受 Spring 管理的组件。
- **@Autowired**：用于自动装配 Spring Bean。
- **@Value**：用于注入配置属性值。
- **@RequestMapping**：用于映射 HTTP 请求路径到 Controller 的处理方法。
- **@GetMapping**、@PostMapping、@PutMapping、@DeleteMapping：简化@RequestMapping 的 GET、POST、PUT 和 DELETE 请求。

另外，一个与配置相关的重要注解是：

- **@Configuration**：用于指定一个类为配置类，其中定义的 bean 会被 Spring 容器管理。通常与@Bean 配合使用，@Bean 用于声明一个 Bean 实例，由 Spring 容器进行管理。

## springboot 怎么开启事务?

在 Spring Boot 中开启事务非常简单，只需在服务层的方法上添加 `@Transactional` 注解即可。

例如，假设我们有一个 UserService 接口，其中有一个保存用户的方法 saveUser()：

```
public interface UserService {
    void saveUser(User user);
}
```

我们希望在这个方法中开启事务，只需在该方法上添加 `@Transactional` 注解，如下所示：

```
public class UserServiceImpl implements UserService {

    @Autowired
    private UserRepository userRepository;

    @Override
    @Transactional
    public void saveUser(User user) {
        userRepository.save(user);
    }
```

```
}
```

这样，当调用 saveUser() 方法时，Spring 就会自动为该方法开启一个事务。如果方法执行成功，事务会自动提交；如果方法执行失败，事务会自动回滚。

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

## Springboot 怎么做到导入就可以直接使用的?

这个主要依赖于自动配置、起步依赖和条件注解等特性。

> 起步依赖

起步依赖是一种特殊的 Maven 或 Gradle 依赖，它将项目所需的一系列依赖打包在一起。例如，`spring-boot-starter-web` 这个起步依赖就包含了 Spring Web MVC、Tomcat 等构建 Web 应用所需的核心依赖。

开发者只需在项目中添加一个起步依赖，Maven 或 Gradle 就会自动下载并管理与之关联的所有依赖，避免了手动添加大量依赖的繁琐过程。

比如，在 `pom.xml` 中添加 `spring-boot-starter-web` 依赖：

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

> 自动配置

Spring Boot 的自动配置机制会根据类路径下的依赖和开发者的配置，自动创建和配置应用所需的 Bean。它通过 `@EnableAutoConfiguration` 注解启用，该注解会触发 Spring Boot 去查找 `META-INF/spring.factories` 文件。

`spring.factories` 文件中定义了一系列自动配置类，Spring Boot 会根据当前项目的依赖情况，选择合适的自动配置类进行加载。例如，如果项目中包含 `spring-boot-starter-web` 依赖，Spring Boot 会加载 `WebMvcAutoConfiguration` 类，该类会自动配置 Spring MVC 的相关组件，如 DispatcherServlet、视图解析器等。

开发者可以通过自定义配置来覆盖自动配置的默认行为。如果开发者在 `application.properties` 或 `application.yml` 中定义了特定的配置，或者在代码中定义了同名的 Bean，Spring Boot 会优先使用开发者的配置。

> 条件注解

条件注解用于控制 Bean 的创建和加载，只有在满足特定条件时，才会创建相应的 Bean。Spring Boot 的自动配置类中广泛使用了条件注解，如 `@ConditionalOnClass`、`@ConditionalOnMissingBean` 等。

比如，`@ConditionalOnClass` 表示只有当类路径中存在指定的类时，才会创建该 Bean。例如，在 `WebMvcAutoConfiguration` 类中，可能会有如下代码：

```
@Configuration
@ConditionalOnClass({ Servlet.class, DispatcherServlet.class, WebMvcConfigurer.class })
public class WebMvcAutoConfiguration {
    // 配置相关的 Bean
}
```

这段代码表示只有当类路径中存在 `Servlet`、`DispatcherServlet` 和 `WebMvcConfigurer` 类时，才会加载 `WebMvcAutoConfiguration` 类中的配置。

## SpringBoot 过滤器和拦截器说一下?

在 Spring Boot 中，过滤器（Filter）和拦截器（Interceptor）是用于处理请求和响应的两种不同机制。

| 特性 | 过滤器（Filter） | 拦截器（Interceptor） |
| --- | --- | --- |
| 规范/框架 | Servlet 规范（`javax.servlet.Filter`） | Spring MVC 框架（`org.springframework.web.servlet.HandlerInterceptor`） |

| 特性 | 过滤器（Filter） | 拦截器（Interceptor） |
| --- | --- | --- |
| 作用范围 | 全局（所有请求、静态资源） | Controller 层（仅拦截 Spring 管理的请求） |
| 执行顺序 | 在 Servlet 之前执行 | 在 DispatcherServlet 之后、Controller 方法前后执行 |
| 依赖注入支持 | 无法直接注入 Spring Bean（需间接获取） | 支持自动注入 Spring Bean |
| 触发时机 | `doFilter()`在请求前/响应后被调用 | `preHandle`、`postHandle`、`afterCompletion`分阶段触发 |
| 适用场景 | 全局请求处理（编码、日志、安全） | 业务逻辑相关的处理（权限、参数校验） |

过滤器是 Java Servlet 规范中的一部分，它可以对进入 Servlet 容器的请求和响应进行预处理和后处理。过滤器通过实现 `javax.servlet.Filter` 接口，并重写其中的 `init`、`doFilter` 和 `destroy` 方法来完成相应的逻辑。当请求进入 Servlet 容器时，会按照配置的顺序依次经过各个过滤器，然后再到达目标 Servlet 或控制器；响应返回时，也会按照相反的顺序再次经过这些过滤器。

拦截器是 Spring 框架提供的一种机制，它可以对控制器方法的执行进行拦截。拦截器通过实现 `org.springframework.web.servlet.HandlerInterceptor` 接口，并重写其中的 `preHandle`、`postHandle` 和 `afterCompletion` 方法来完成相应的逻辑。当请求到达控制器时，会先经过拦截器的 `preHandle` 方法，如果该方法返回 `true`，则继续执行后续的控制器方法和其他拦截器；在控制器方法执行完成后，会调用拦截器的 `postHandle` 方法；最后，在请求处理完成后，会调用拦截器的 `afterCompletion` 方法。

过滤器和拦截器的区别如下：

- **所属规范**：过滤器是 Java Servlet 规范的一部分，而拦截器是 Spring 框架提供的机制。
- **执行顺序**：过滤器在请求进入 Servlet 容器后，在到达目标 Servlet 或控制器之前执行；拦截器在请求到达控制器之后，在控制器方法执行前后执行。
- **使用范围**：过滤器可以对所有类型的请求进行过滤，包括静态资源请求；拦截器只能对 Spring MVC 控制器的请求进行拦截。
- **功能特性**：过滤器主要用于对请求和响应进行预处理和后处理，如字符编码处理、请求日志记录等；拦截器可以更细粒度地控制控制器方法的执行，如权限验证、性能监控等。

# Mybatis

## 与传统的 JDBC 相比，MyBatis 的优点?

- 基于 SQL 语句编程，相当灵活，不会对应用程序或者数据库的现有设计造成任 何影响，SQL 写在 XML 里，解除 sql 与程序代码的耦合，便于统一管理；提供 XML 标签，支持编写动态 SQL 语句，并可重用。
- 与 JDBC 相比，减少了 50%以上的代码量，消除了 JDBC 大量冗余的代码，不 需要手动开关连接；
- 很好的与各种数据库兼容，因为 MyBatis 使用 JDBC 来连接数据库，所以只要 JDBC 支持的数据库 MyBatis 都支持。
- 能够与 Spring 很好的集成，开发效率高

- 提供映射标签，支持对象与数据库的 ORM 字段关系映射；提供对象关系映射 标签，支持对象关系组件维护。

## MyBatis 觉得在哪方面做的比较好?

MyBatis 在 **SQL 灵活性、动态 SQL 支持、结果集映射**和**与 Spring 整合**方面表现卓越，尤其适合重视 SQL 可控性的项目。

- SQL 与代码解耦，灵活可控：MyBatis 允许开发者直接编写和优化 SQL，相比全自动 ORM（如 Hibernate），MyBatis 让开发者明确知道每条 SQL 的执行逻辑，便于性能调优。

```
<!-- 示例：XML 中定义 SQL -->
<select id="findUserWithRole" resultMap="userRoleMap">
    SELECT u.*, r.role_name
    FROM user u
    LEFT JOIN user_role ur ON u.id = ur.user_id
    LEFT JOIN role r ON ur.role_id = r.id
    WHERE u.id = #{userId}
</select>
```

- 动态 SQL 的强大支持：比如可以动态拼接 SQL，通过 `<if>`，`<choose>`，`<foreach>` 等标签动态生成 SQL，避免 Java 代码中繁琐的字符串拼接。

```
<select id="searchUsers" resultType="User">
    SELECT * FROM user
    <where>
        <if test="name != null">AND name LIKE #{name}</if>
        <if test="status != null">AND status = #{status}</if>
    </where>
</select>
```

- 自动映射与自定义映射结合：自动将查询结果字段名与对象属性名匹配（如驼峰转换）。

```
<resultMap id="userRoleMap" type="User">
    <id property="id" column="user_id"/>
    <result property="name" column="user_name"/>
    <collection property="roles" ofType="Role">
        <result property="roleName" column="role_name"/>
```

```
    </collection>
</resultMap>
```

- 插件扩展机制：可编写插件拦截 SQL 执行过程，实现分页、性能监控、SQL 改写等通用逻辑。

```
@Intercepts({
    @Signature(type=Executor.class, method="query", args={...})
})
public class PaginationPlugin implements Interceptor {
    // 实现分页逻辑
}
```

- 与 Spring 生态无缝集成：通过 `@MapperScan` 快速扫描 Mapper 接口，结合 Spring 事务管理，配置简洁高效。

```
@Configuration
@MapperScan("com.example.mapper")
public class MyBatisConfig {
    // 数据源和 SqlSessionFactory 配置
}
```

## 还记得 JDBC 连接数据库的步骤吗?

使用 Java JDBC 连接数据库的一般步骤如下：

1. **加载数据库驱动程序**：在使用 JDBC 连接数据库之前，需要加载相应的数据库驱动程序。可以通过 Class.forName("com.mysql.jdbc.Driver") 来加载 MySQL 数据库的驱动程序。不同数据库的驱动类名会有所不同。
2. **建立数据库连接**：使用 DriverManager 类的 getConnection(url, username, password) 方法来连接数据库，其中 url 是数据库的连接字符串（包括数据库类型、主机、端口等）、username 是数据库用户名，password 是密码。
3. **创建 Statement 对象**：通过 Connection 对象的 createStatement() 方法创建一个 Statement 对象，用于执行 SQL 查询或更新操作。

4. **执行 SQL 查询或更新操作**：使用 Statement 对象的 executeQuery(sql) 方法来执行 SELECT 查询操作，或者使用 executeUpdate(sql) 方法来执行 INSERT、UPDATE 或 DELETE 操作。
5. **处理查询结果**：如果是 SELECT 查询操作，通过 ResultSet 对象来处理查询结果。可以使用 ResultSet 的 next() 方法遍历查询结果集，然后通过 getXXX() 方法获取各个字段的值。
6. **关闭连接**：在完成数据库操作后，需要逐级关闭数据库连接相关对象，即先关闭 ResultSet，再关闭 Statement，最后关闭 Connection。

以下是一个简单的示例代码：

```
import java.sql.*;


public class Main {
    public static void main(String[] args) {
        try {
            // 加载数据库驱动程序
            Class.forName("com.mysql.cj.jdbc.Driver");


            // 建立数据库连接
            Connection connection =
DriverManager.getConnection("jdbc:mysql://localhost:3306/mydatabase", "username", "password");


            // 创建 Statement 对象
            Statement statement = connection.createStatement();


            // 执行 SQL 查询
            ResultSet resultSet = statement.executeQuery("SELECT * FROM mytable");


            // 处理查询结果
            while (resultSet.next()) {
              // 处理每一行数据
            }


            // 关闭资源
            resultSet.close();
```

```
            statement.close();
            connection.close();
        } catch (ClassNotFoundException e) {
            e.printStackTrace();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

请注意，在实际应用中，需要进行异常处理以确保资源的正确释放，以及使用 try-with-resources 来简化代码和确保资源的及时关闭。

## 如果项目中要用到原生的 mybatis 去查询，该怎样写?

步骤概述：

1. **配置 MyBatis：** 在项目中配置 MyBatis 的数据源、SQL 映射文件等。
2. **创建实体类：** 创建用于映射数据库表的实体类。
3. **编写 SQL 映射文件：** 创建 XML 文件，定义 SQL 语句和映射关系。
4. **编写 DAO 接口：** 创建 DAO 接口，定义数据库操作的方法。
5. **编写具体的 SQL 查询语句：** 在 DAO 接口中定义查询方法，并在 XML 文件中编写对应的 SQL 语句。
6. **调用查询方法：** 在服务层或控制层调用 DAO 接口中的方法进行查询。

详细步骤：

1. **配置 MyBatis：** 在配置文件中配置数据源、MyBatis 的 Mapper 文件位置等信息。
2. **创建实体类：** 创建与数据库表对应的实体类，字段名和类型需与数据库表保持一致。

```
public class User {
    private Long id;
    private String username;
    private String email;
    // Getters and setters
}
```

1. **编写 SQL 映射文件：** 在 resources 目录下创建 XML 文件，定义 SQL 语句和映射关系。

```
<!-- userMapper.xml -->
<mapper namespace="com.example.dao.UserMapper">
    <select id="selectUserById" resultType="com.example.model.User">
        SELECT * FROM users WHERE id = #{id}
    </select>
</mapper>
```

1. **编写 DAO 接口：** 创建 DAO 接口，定义查询方法。

```
public interface UserMapper {
    User selectUserById(Long id);
}
```

1. **编写具体的 SQL 查询语句：** 在 XML 文件中编写对应的 SQL 语句。
2. **调用查询方法：** 在服务层或控制层中调用 DAO 接口中的方法进行查询。

```
// 在 Service 层中调用
User user = userMapper.selectUserById(1);
```

通过以上步骤，你可以利用原生的 MyBatis 框架来进行数据库查询操作。请确保配置正确、SQL 语句准确并与数据库字段匹配，以确保查询的准确性和高效性。

## Mybatis 里的 # 和 $ 的区别?

- Mybatis 在处理 #{} 时，会创建预编译的 SQL 语句，将 SQL 中的 #{} 替换为 ? 号，在执行 SQL 时会为预编译 SQL 中的占位符（?）赋值，调用 PreparedStatement 的 set 方法来赋值，预编译的 SQL 语句执行效率高，并且可以防止 SQL 注入，提供更高的安全性，适合传递参数值。
- Mybatis 在处理 ${} 时，只是创建普通的 SQL 语句，然后在执行 SQL 语句时 MyBatis 将参数直接拼入到 SQL 里，不能防止 SQL 注入，因为参数直接拼接到 SQL 语句中，如果参数未经过验证、过滤，可能会导致安全问题。

## MybatisPlus 和 Mybatis 的区别?

MybatisPlus 是一个基于 MyBatis 的增强工具库，旨在简化开发并提高效率。以下是 MybatisPlus 和 MyBatis 之间的一些主要区别：

- **CRUD 操作**：MybatisPlus 通过继承 BaseMapper 接口，提供了一系列内置的快捷方法，使得 CRUD 操作更加简单，无需编写重复的 SQL 语句。
- **代码生成器**：MybatisPlus 提供了代码生成器功能，可以根据数据库表结构自动生成实体类、Mapper 接口以及 XML 映射文件，减少了手动编写的工作量。
- **通用方法封装**：MybatisPlus 封装了许多常用的方法，如条件构造器、排序、分页查询等，简化了开发过程，提高了开发效率。
- **分页插件**：MybatisPlus 内置了分页插件，支持各种数据库的分页查询，开发者可以轻松实现分页功能，而在传统的 MyBatis 中，需要开发者自己手动实现分页逻辑。
- **多租户支持**：MybatisPlus 提供了多租户的支持，可以轻松实现多租户数据隔离的功能。
- **注解支持**：MybatisPlus 引入了更多的注解支持，使得开发者可以通过注解来配置实体与数据库表之间的映射关系，减少了 XML 配置文件的编写。

### MyBatis 运用了哪些常见的设计模式?

- 建造者模式（Builder），如：SqlSessionFactoryBuilder、XMLConfigBuilder、XMLMapperBuilder、XMLStatementBuilder、CacheBuilder 等；
- 工厂模式，如：SqlSessionFactory、ObjectFactory、MapperProxyFactory；
- 单例模式，例如 ErrorContext 和 LogFactory；
- 代理模式，Mybatis 实现的核心，比如 MapperProxy、ConnectionLogger，用的 jdk 的动态代理；还有 executor.loader 包使用了 cglib 或者 javassist 达到延迟加载的效果；
- 组合模式，例如 SqlNode 和各个子类 ChooseSqlNode 等；
- 模板方法模式，例如 BaseExecutor 和 SimpleExecutor，还有 BaseTypeHandler 和所有的子类例如 IntegerTypeHandler；
- 适配器模式，例如 Log 的 Mybatis 接口和它对 jdbc、log4j 等各种日志框架的适配实现；
- 装饰者模式，例如 Cache 包中的 cache.decorators 子包中等各个装饰者的实现；
- 迭代器模式，例如迭代器模式 PropertyTokenizer；

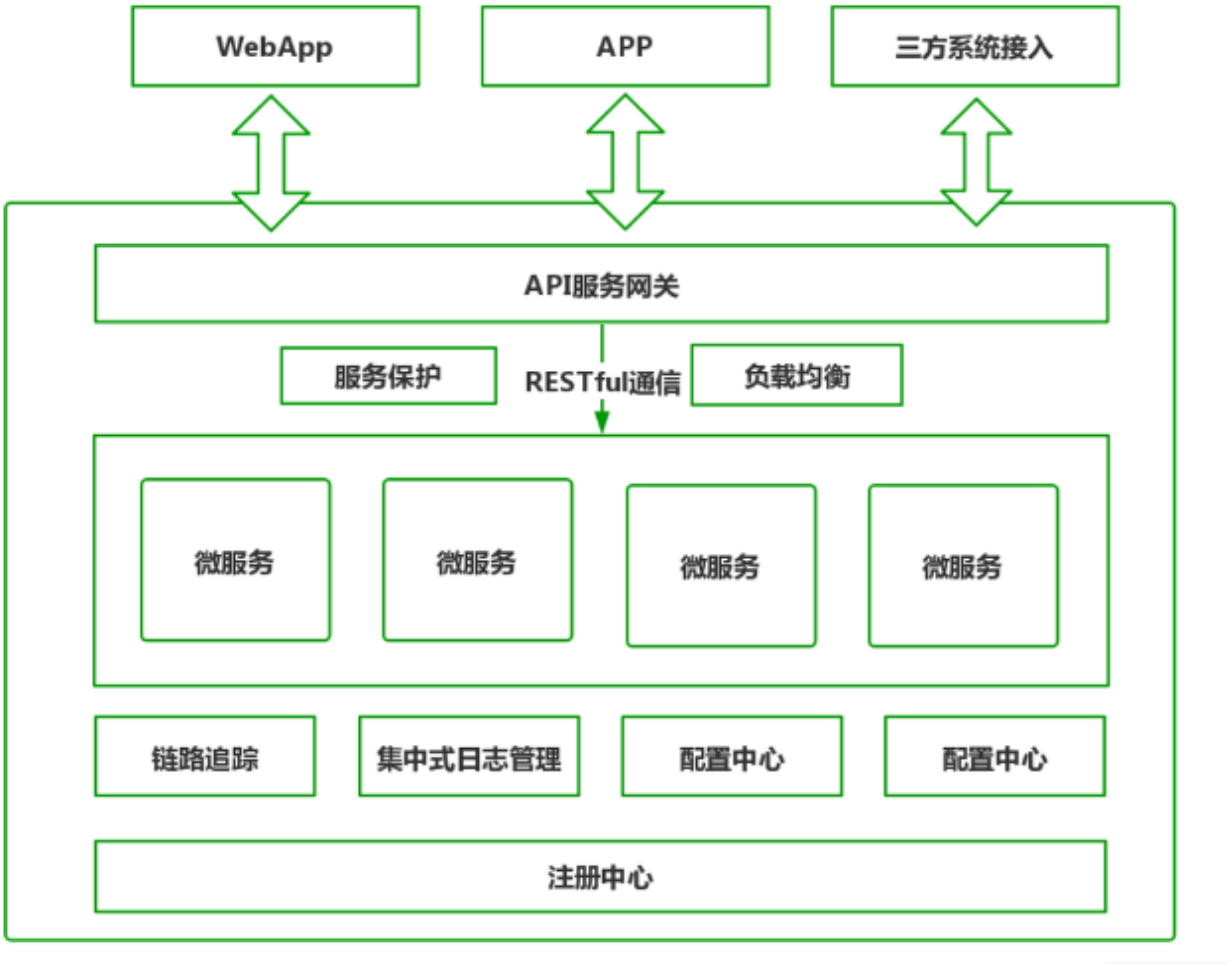
## SpringCloud

### 了解 SpringCloud 吗，说一下他和 SpringBoot 的区别

Spring Boot 是用于构建单个 Spring 应用的框架，而 Spring Cloud 则是用于构建分布式系统中的微服务架构的工具，Spring Cloud 提供了服务注册与发现、负载均衡、断路器、网关等功能。

两者可以结合使用，通过 Spring Boot 构建微服务应用，然后用 Spring Cloud 来实现微服务架构中的各种功能。

## 用过哪些微服务组件?

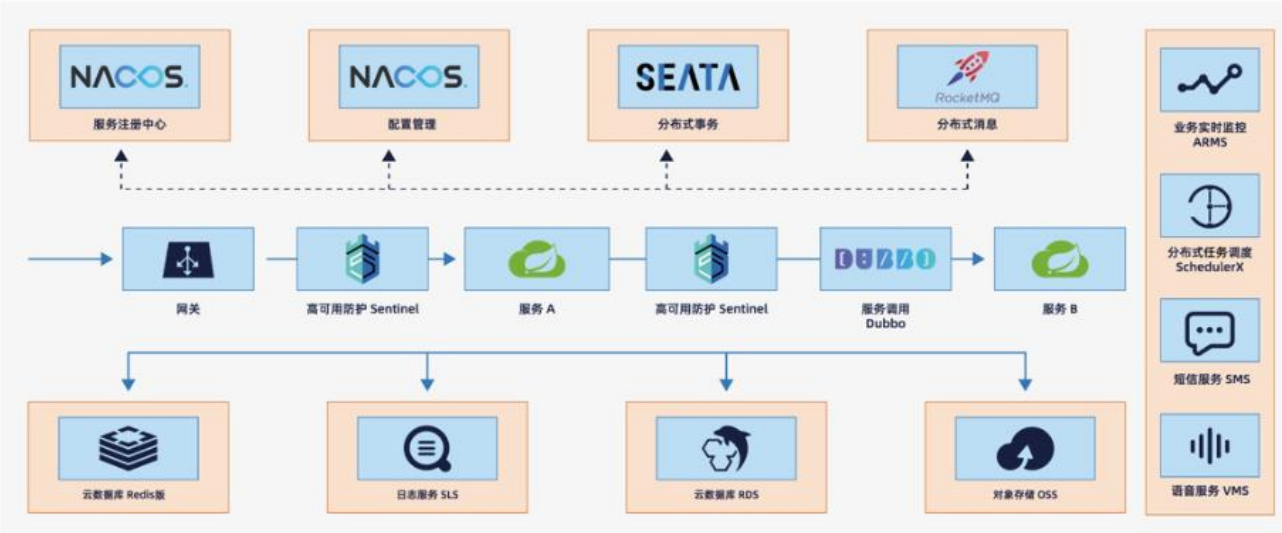



微服务常用的组件：

- **注册中心**：注册中心是微服务架构最核心的组件。它起到的作用是对新节点的注册与状态维护，**解决了「如何发现新节点以及检查各节点的运行状态的问题」**。微服务节点在启动时会将自己的服务名称、IP、端口等信息在注册中心登记，注册中心会定时检查该节点的运行状态。注册中心通常会采用心跳机制最大程度保证已登记过的服务节点都是可用的。
- **负载均衡**：负载均衡**解决了「如何发现服务及负载均衡如何实现的问题」**，通常微服务在互相调用时，并不是直接通过 IP、端口进行访问调用。而是先通过服务名在注册中心查询该服务拥有哪些节点，注册中心将该服务可用节点列表返回给服务调用者，这个过程叫服务发现，因服务高可用的要求，服务调用者会接收到多个节点，必须要从中进行选择。因此服务调用者一端必须内置负载均衡器，通过负载均衡策略选择合适的节点发起实质性的通信请求。

- **服务通信**：服务通信组件解决了「**服务间如何进行消息通信的问题**」，服务间通信采用轻量级协议，通常是 HTTP RESTful 风格。但因为 RESTful 风格过于灵活，必须加以约束，通常应用时对其封装。例如在 SpringCloud 中就提供了 Feign 和 RestTemplate 两种技术屏蔽底层的实现细节，所有开发者都是基于封装后统一的 SDK 进行开发，有利于团队间的相互合作。
- **配置中心**：配置中心主要解决了「**如何集中管理各节点配置文件的问题**」，在微服务架构下，所有的微服务节点都包含自己的各种配置文件，如 jdbc 配置、自定义配置、环境配置、运行参数配置等。要知道有的微服务可能可能有几十个节点，如果将这些配置文件分散存储在节点上，发生配置更改就需要逐个节点调整，将给运维人员带来巨大的压力。配置中心便由此而生，通过部署配置中心服务器，将各节点配置文件从服务中剥离，集中转存到配置中心。一般配置中心都有 UI 界面，方便实现大规模集群配置调整。
- **集中式日志管理**：集中式日志主要是解决了「**如何收集各节点日志并统一管理的问题**」。微服务架构默认将应用日志分别保存在部署节点上，当需要对日志数据和操作数据进行数据分析和数据统计时，必须收集所有节点的日志数据。那么怎么高效收集所有节点的日志数据呢？业内常见的方案有 ELK、EFK。通过搭建独立的日志收集系统，定时抓取各节点增量日志形成有效的统计报表，为统计和分析提供数据支撑。
- 分布式链路追踪：分布式链路追踪解决了「**如何直观的了解各节点间的调用链路的问题**」。系统中一个复杂的业务流程，可能会出现连续调用多个微服务，我们需要了解完整的业务逻辑涉及的每个微服务的运行状态，通过可视化链路图展现，可以帮助开发人员快速分析系统瓶颈及出错的服务。
- **服务保护**：服务保护主要是解决了「**如何对系统进行链路保护，避免服务雪崩的问题**」。在业务运行时，微服务间互相调用支撑，如果某个微服务出现高延迟导致线程池满载，或是业务处理失败。这里就需要引入服务保护组件来实现高延迟服务的快速降级，避免系统崩溃。

SpringCloud Alibaba 实现的微服务架构：



- SpringCloud Alibaba 中使用 **Alibaba Nacos** 组件实现**注册中心**，Nacos 提供了一组简单易用的特性集，可快速实现动态服务发现、服务配置、服务元数据及流量管理。
- SpringCloud Alibaba 使用 **Nacos 服务端均衡**实现负载均衡，与 Ribbon 在调用端负载不同，Nacos 是在服务发现的同时利用负载均衡返回服务节点数据。
- SpringCloud Alibaba 使用 **Netflix Feign** 和 **Alibaba Dubbo** 组件来实现服务通行，前者与 SpringCloud 采用了相同的方案，后者则是对自家的 **RPC 框架 Dubbo** 也给予支持，为服务间通信提供另一种选择。
- SpringCloud Alibaba 在 **API 服务网关**组件中，使用与 SpringCloud 相同的组件，即：**SpringCloud Gateway**。
- SpringCloud Alibaba 在配置中心组件中使用 **Nacos 内置配置中心**，Nacos 内置的配置中心，可将配置信息**存储保存在指定数据库**中
- SpringCloud Alibaba 在原有的 **ELK 方案**外，还可以使用阿里云日志服务（LOG）实现日志集中式管理。
- SpringCloud Alibaba 在**分布式链路组件**中采用与 SpringCloud 相同的方案，即：**Sleuth/Zipkin Server**。
- SpringCloud Alibaba 使用 **Alibaba Sentinel** 实现系统保护，Sentinel 不仅功能更强大，实现系统保护比 Hystrix 更优雅，而且还拥有更好的 UI 界面。

## 负载均衡有哪些算法?

- 简单轮询：将请求按顺序分发给后端服务器上，不关心服务器当前的状态，比如后端服务器的性能、当前的负载。
- 加权轮询：根据服务器自身的性能给服务器设置不同的权重，将请求按顺序和权重分发给后端服务器，可以让性能高的机器处理更多的请求
- 简单随机：将请求随机分发给后端服务器上，请求越多，各个服务器接收到的请求越平均
- 加权随机：根据服务器自身的性能给服务器设置不同的权重，将请求按各个服务器的权重随机分发给后端服务器
- 一致性哈希：根据请求的客户端 ip、或请求参数通过哈希算法得到一个数值，利用该数值取模映射出对应的后端服务器，这样能保证同一个客户端或相同参数的请求每次都使用同一台服务器
- 最小活跃数：统计每台服务器上当前正在处理的请求数，也就是请求活跃数，将请求分发给活跃数最少的后台服务器

## 如何实现一直均衡给一个用户?

可以通过「一致性哈希算法」来实现，根据请求的客户端 ip、或请求参数通过哈希算法得到一个数值，利用该数值取模映射出对应的后端服务器，这样能保证同一个客户端或相同参数的请求每次都使用同一台服务器。

## 介绍一下服务熔断

服务熔断是应对微服务雪崩效应的一种**链路保护机制，类似股市、保险丝**。

比如说，微服务之间的数据交互是通过远程调用来完成的。服务 A 调用服务，服务 B 调用服务 c，某一时间链路上对服务 C 的调用响应时间过长或者服务 C 不可用，随着时间的增长，对服务 C 的调用也越来越多，然后服务 C 崩溃了，但是链路调用还在，对服务 B 的调用也在持续增多，然后服务 B 崩溃，随之 A 也崩溃，导致雪崩效应。

服务熔断是应对雪崩效应的一种微服务链路保护机制。例如在高压电路中，如果某个地方的电压过高，熔断器就会熔断，对电路进行保护。同样，在微服务架构中，熔断机制也是起着类似的作用。当调用链路的某个微服务不可用或者响应时间太长时，会进行服务熔断，不再有该节点微服务的调用，快速返回错误的响应信息。当检测到该节点微服务调用响应正常后，恢复调用链路。

所以，服务熔断的作用类似于我们家用的保险丝，当某服务出现不可用或响应超时的情况时，为了防止整个系统出现雪崩，暂时停止对该服务的调用。

在 Spring Cloud 框架里，熔断机制通过 Hystrix 实现。Hystrix 会监控微服务间调用的状况，当失败的调用到一定阈值，缺省是 5 秒内 20 次调用失败，就会启动熔断机制。

## 介绍一下服务降级

服务降级一般是指在服务器压力剧增的时候，根据实际业务使用情况以及流量，对一些服务和页面有策略的不处理或者用一种简单的方式进行处理，从而**释放服务器资源的资源以保证核心业务的正常高效运行。**

服务器的资源是有限的，而请求是无限的。在用户使用即并发高峰期，会影响整体服务的性能，严重的话会导致宕机，以至于某些重要服务不可用。故高峰期为了保证核心功能服务的可用性，就需要对某些服务降级处理。可以理解为舍小保大

服务降级是从整个系统的负荷情况出发和考虑的，对某些负荷会比较高的情况，为了预防某些功能（业务场景）出现负荷过载或者响应慢的情况，在其内部暂时舍弃对一些非核心的接口和数据的请求，而直接返回一个提前准备好的 fallback（退路）错误处理信息。这样，虽然提供的是一个有损的服务，但却保证了整个系统的稳定性和可用性。